



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSELMÉLET ÉS SZOFTVERTECHNOLÓGIAI
TANSZÉK

Online Többjátékos Snapszer

Témavezető:

Gaál Bence
egyetemi adjunktus

Szerző:

Lesták András
programtervező informatikus BSc

Budapest, 2026

Tartalomjegyzék

1. Bevezetés	4
1.1. Témabejelentő	4
1.2. A dolgozat felépítése	5
1.3. A snapszer szabályai	6
1.3.1. Kártyák értéke és a játék célja	6
1.3.2. A játék menete	6
1.3.3. A takarás (zárás) és a kényszerek	7
1.3.4. Speciális szabályok	7
1.3.5. Végző pontozás (Meccspontok)	8
1.4. Motiváció	8
2. Felhasználói dokumentáció	9
2.1. Követelmények	9
2.2. Rövid weboldalleírás	9
2.2.1. A játékmenet az alábbi folyamat mentén zajlik:	9
2.3. Oldaltérkép és navigáció	10
2.3.1. Oldaltérkép	11
2.3.2. Navigáció	11
2.3.3. Grafikai elemek és erőforrások	15
2.4. A játék indítása	15
2.4.1. Név megadása	15
2.4.2. Játék keresése	15
2.4.3. Várakozás és kezdés	16
2.4.4. Játékkeresés bejelentkezett felhasználóként	16
2.5. Regisztráció és bejelentkezés	17
2.5.1. Fiók létrehozása (Regisztráció)	17
2.5.2. Bejelentkezés és annak előnyei	18

2.6.	Játékmenet	19
2.6.1.	Kártyakijátszás és szabályosság vizsgálata	19
2.6.2.	Játékosállapot és pontszámok kijelzése	20
2.6.3.	Saját ütések megtekintése	22
2.7.	Speciális lépések és stratégiai lehetőségek	23
2.7.1.	Húsz és negyven bemondása	23
2.7.2.	Aducsere	23
2.7.3.	Takarás (A pakli lezárása)	24
3.	Fejlesztői dokumentáció	26
3.0.1.	Célközönség és a szoftver célkitűzései	26
3.1.	Implementációt megelőző rendszertervek	27
3.1.1.	Használatieset-diagram (Use-case diagram)	27
3.1.2.	A felhasználói felület tervei (Figma)	29
3.1.3.	Komponens diagramok	31
3.1.4.	Osztálydiagram (UML)	32
3.1.5.	Egyed-kapcsolat diagram (ER diagram)	33
3.2.	A szoftver belső struktúrája és adatábrázolása	34
3.2.1.	Moduláris felépítés és kódminőség	34
3.2.2.	Adatábrázolási döntések (Sequelize ORM)	35
3.2.3.	Kliensoldali architektúra és állapotkezelés	35
3.2.4.	Fizikai komponens-telepítési diagram és infrastruktúra	36
3.3.	A megvalósult rendszerstruktúra	37
3.3.1.	Végleges osztálydiagram (UML)	38
3.3.2.	Osztályok	38
3.3.3.	Végleges adatbázis séma (ER diagram)	39
3.3.4.	Az adatbázis táblái	40
3.4.	Backend architektúra és infrastruktúra	41
3.4.1.	Fontosabb metódusok dokumentációja	41
3.4.2.	A szerveroldali belépési pont (server.js)	43
3.4.3.	REST API végpontok dokumentációja	44
3.4.4.	Rendszerezemények naplózása (Logging)	45
3.4.5.	Alkalmazott szoftverkönyvtárak és függőségek	46
3.5.	Fejlesztői környezet kialakítása és futtatás helyi környezetben	47

3.5.1.	Előfeltételek	47
3.5.2.	A Backend környezet beállítása	48
3.5.3.	A Frontend környezet beállítása	49
3.6.	Rendszertesztelés és a megvalósult használati esetek	49
3.6.1.	Egység tesztek	54
3.6.2.	Tesztelés által befolyásolt döntések	58
4.	Összegzés	60
4.1.	Továbbfejlesztési lehetőségek	60
4.1.1.	Többjátékos (3 és 4 fős) játékmódok integrálása	60
4.1.2.	Egyjátékos mód: Játék "Bot" ellen	61
	Irodalomjegyzék	62
	Ábrajegyzék	64
	Táblázatjegyzék	66
	Algoritmusjegyzék	67

1. fejezet

Bevezetés

A modern full-stack fejlesztés egyik legösszetettebb feladata a valós idejű, több-játékos rendszerek létrehozása, ahol a kliensoldali élmény és a szerveroldali biztonság egyaránt kritikus. Jelen dolgozat ezen kihívásokat mutatja be a Snapszer kártyajáték webes adaptációján keresztül. A fejezet elsőként a fejlesztés céljait rögzítő témabejelentőt tartalmazza, majd részletezi a játék azon szabályait, amelyek az implementáció logikai alapját képezik.

1.1. Témabejelentő

A dolgozatom témája egy Online Többjátékos Snapszer Webes Játék megtervezése és implementálása. A célom, hogy egy letisztult modern játékfelületet létrehozzak, ami optimális játékelményt biztosít. A projekt bemutatja, hogy hogyan oldható meg egy valós idejű szinkronizációs játék webes környezetben.

A snapszer egy kiemelten népszerű kártyajáték itthon, melynek online verziója magas követelményeket támaszt a valós idejű interakció és a precíz játékszabály validáció terén. Míg asztali és mobil alkalmazások léteznek, webes környezetben még nincsen letisztult megvalósítása a játéknak.

A dolgozatom fő feladata, hogy gyakorlati példán keresztül bemutassa a full-stack webfejlesztés lehetőségeit. Egy olyan összetett rendszert valósít meg, amelynél elengedhetetlen a minimális késleltetéssel járó kommunikáció és a szerver oldali, biztonságos játékállapot kezelés. A választott JavaScript technológiai stack (React.js, Node.js, Express.js) azért ideális, mert egységes és hatékony alapot biztosít mind a

felhasználói felület, mind a háttérben futó játékszervert komplex kihívásainak megoldásához és egyetlen nyelvet használ az egész stack.

1.2. A dolgozat felépítése

A dolgozat négy fő fejezetből, valamint az Irodalomjegyzékből, az Ábrajegyzékből és a Táblázatok jegyzékéből áll.

1. Bevezetés

Itt mutatom be a témaválasztásom motivációját, a snapszer kártyajáték részletes szabályait, valamint a fejlesztés alapjául szolgáló témabejelentő is itt kapott helyet.

2. Felhasználói dokumentáció

Ez egy kézikönyv a játékosok számára, amelyből megismerhetik a program futtatásához szükséges követelményeket és az alkalmazás funkcióit. Itt részletesen bemutatom a weboldal felépítését, az oldaltérképet, az automatikus meccskeresés és a játék indításának folyamatát. Továbbá a fejezet tartalmaz egy lépésről lépésre haladó útmutatót is, amely a tényleges játékmenetet és a virtuális kártyaasztal használatát mutatja be.

3. Fejlesztői dokumentáció

Ez a program technikai leírása, amely az alkalmazás egy későbbi módosítása vagy bővítése esetén nyújt segítséget. Itt található a rendszer architektúrájának bemutatása, az alkalmazott webes technológiák (kliens és szerver oldal) ismertetése, az adatbázis felépítése, valamint a valós idejű többjátékos szinkronizáció megvalósításának részletei.

4. Összegzés

A dolgozat lezárásaként értékelem az elvégzett munkát, összegzem a fejlesztés során szerzett tapasztalatokat, és felvázolom az alkalmazás lehetséges jövőbeli továbbfejlesztési irányait.

Irodalomjegyzék

Itt hivatkozom a munkám során használt cikkekre, hivatalos technológiai dokumentációkra és eszközökre.

Ábrajegyzék

A dolgozatban előforduló ábrák, diagramok és képernyőképek gyűjteménye.

Táblázatok jegyzéke

A dolgozatban előforduló táblázatok gyűjteménye.

1.3. A snapszer szabályai

A Snapszer (vagy 66) egy 20 lapos magyar kártyapaklival játszott, magas stratégiai mélységű ütésváltós játék. A szoftveres megvalósítás során az alábbi kétjátékos szabályrendszert vettem alapul.

1.3.1. Kártyák értéke és a játék célja

A játékban csak az Ász, a Tizes, a Király, a Felső és az Alsó lapok vesznek részt. A körök (osztások) elsődleges célja elérni a 66 pontot az ütések és bemondások értékéből, amellyel a játékos meccspontokat szerezhet. Egy teljes meccs általában 7 meccspontig tart.

- **Ász:** 11 pont, **Tizes:** 10 pont.
- **Király:** 4 pont, **Felső:** 3 pont, **Alsó:** 2 pont.

1.3.2. A játék menete

A játékosok 5-5 lapot kapnak, a húzópakli (talon) legfelső lapját pedig felfordítják, ez határozza meg az adu (tromf) színét.

- **Nyílt játék (Talon fázis):** Amíg a húzható pakli el nem fogy, vagy le nem zárják, nem kötelező sem színre színt tenni, sem ütni. A hívott lapra bármilyen kártya dobható.
- **Ütések és az adu ereje:** Az ütést a hívott szín nagyobb értékű lapja, vagy (ha ilyet nem tettek) az adu viszi el. Az adu minden más színt felülbírál, függetlenül a másik lap értékétől.
- **Laphúzás:** Minden ütés után mindkét játékos húz egy új lapot a talonból, hogy újra 5 lapjuk legyen. Mindig az ütést megnyerő játékos húz először.

1.3.3. A takarás (zárás) és a kényszerek

A takarás a játék legkomplexebb stratégiai eleme, amely azonnal végjátékot eredményez.

- **A takarás feltételei:** Kizárólag a hívásra jogosult (soron lévő) játékos dönthet a talon lezárása mellett, amennyiben a pakliban még van legalább 3 lap (a lefordított lapok és a nyitott adu).
- **A takarás menete:** A játékos a nyitott adu kártyát keresztbe fordítja a paklin. Ekkor a húzás lehetősége mindkét fél számára megszűnik, és a kört a kézben maradt lapokkal kell befejezni. A védekező játékos addig megszerzett pontjai a végső elszámoláshoz „befagynak”. A takaró játékos vállalja, hogy a maradék lapjaival eléri a 66 pontot.
- **Kényszerek:** A takarás pillanatától (vagy amint a talon magától elfogy) szigorú szín és ütésekenyszer lép életbe a védekező játékosra, az alábbi prioritási sorrendben:
 1. Kötelező a hívott színnel megegyező lapot tenni, ÉS ha van belőle nagyobb, kötelező felülütni (megnyerni az ütést).
 2. Ha van hívott szín, de csak kisebb, azt kell tenni (aláarakás).
 3. Ha nincs hívott szín, kötelező adut tenni (adukényszer).
 4. Kizárólag akkor lehet más színű (szemét) lapot eldobni, ha sem a hívott színből, sem aduból nincs egyetlen darab sem a játékos kezében.

1.3.4. Speciális szabályok

- **Bemondások:** Ha egy játékos kezében azonos színből Király és Felső is található, híváskor bemondhatja azt. Ez normál szín esetén „húsz” (20 pont), adu színben „negyven” (40 pont). A pontok megadásának feltétele, hogy a játékos a kör során legalább egy ütést szerezzen.
- **Adu csere:** Az adu alsó birtokosa (amennyiben ő jön hívásra) kicserélheti lapját a talon alatt lévő nyitott adu kártyára. **Kivétel:** Takarás megtörténte után ez a csere már szabálytalan és nem hajtható végre.

1.3.5. Végő pontozás (Meccspontok)

A játszma (egy kör) végén a nyertes a következőképpen kap meccspontokat az ellenfél (védekező) elért ütéspontjai alapján:

- **3 pont:** Ha az ellenfélnek egyáltalán nincs ütése (0 pont).
- **2 pont:** Ha az ellenfél 33 pont alatt maradt.
- **1 pont:** Ha az ellenfél elérte a 33 pontot.
- **Takarás bukása:** Ha a takaró játékos a kényszerek ellenére sem éri el a 66 pontot (vagy az ellenfél hamarabb eléri azt), a takaró elbukta a kört. Ekkor a védekező játékos kap 2 meccspontot (vagy 3-at, ha a takaró pont nélkül maradt).
- **Utolsó ütés:** Ha normál játék során a talon elfogy, és a kezben lévő lapok lejátszása végéig egyik fél sem éri el a 66 pontot, automatikusan az utolsó ütést elvivő játékos nyeri a kört.

1.4. Motiváció

A projekt megvalósítását elsősorban az motiválta, hogy a snapszer egy nagy múltú és rendkívül népszerű kártyajáték, amely jelenleg nem rendelkezik modern, böngészőből azonnal játszható, minőségi adaptációval. Egy olyan platform létrehozása volt a célom, amely letöltés, telepítés és akár regisztráció nélkül, letisztult felülettel teszi elérhetővé ezt a játékot a szélesebb közönség számára, tiszteletben tartva a tradicionális szabályokat.

Szakmai oldalról egy valós idejű, többjátékos rendszer felépítése kiemelkedő kihívást jelentett, ami tökéletes lehetőséget biztosított a modern webes technológiák elmélyítésére. A kliens és szerver közötti folyamatos szinkronizáció, a biztonságos állapotkezelés és a reszponzív felület kialakítása révén a projekt komplex keretet ad a full-stack JavaScript fejlesztői képességem alkalmazásához és fejlődéséhez.

2. fejezet

Felhasználói dokumentáció

2.1. Követelmények

	Követelmény
<i>Eszköz</i>	Személyi számítógép vagy laptop
<i>Hálózati elérhetőség</i>	Folyamatos stabil internetkapcsolat
<i>Kijelző méret</i>	Minimum 13"
<i>Böngésző</i>	Modern webböngésző (pl. Chrome vagy Safari)

2.1. táblázat. A program hardver és szoftver követelményei

2.2. Rövid weboldalleírás

A webalkalmazás kezdőképernyője letisztult navigációt biztosít, ahol a játékszabályok egy dedikált aloldalon részletesen áttanulmányozhatók. A rendszer többféle hozzáférési szintet kínál: a felhasználók választhatják a regisztrációhoz kötött bejelentkezést, de lehetőség van bejelentkezés nélküli, vendégként történő játékra is. A bejelentkezési felületről a még fiókkal nem rendelkező látogatók könnyedén átnavigálhatnak a regisztrációs űrlapra. A sikeresen bejelentkezett felhasználók számára a főoldalon elérhetővé válik egy extra funkció, amely a személyes statisztikák megtekintésére szolgál.

2.2.1. A játékmenet az alábbi folyamat mentén zajlik:

- **Meccskeresés:** A mérkőzés indításához vendégként a név megadása, azonosított profil esetén pedig a bejelentkezés szükséges. A párkeresési folyamat a

"**Játék Keresése**" gomb aktiválásával indul, amint a rendszer megtalálja az ellenfelet, a mérkőzés automatikusan kezdetét veszi.

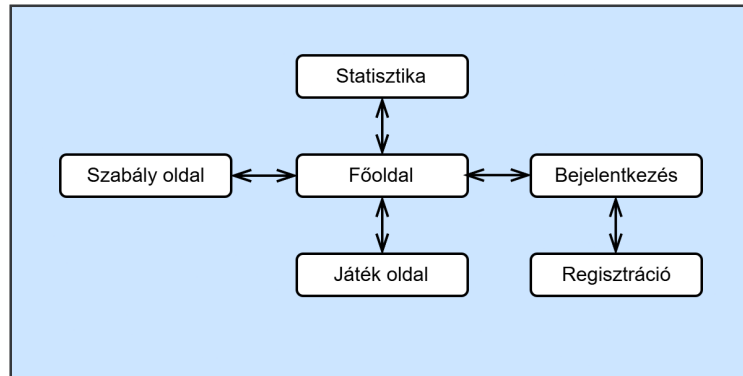
- **Játékmechanika:** A grafikus felület vizuálisan kiemeli az éppen aktív (soron lévő) játékost, a lépésjog pedig minden kártyakijátszás követően automatikusan átkerül az ellenfélhez. A kártyák kijátszása intuitív módon, egy egyszerű kattintással történik, melynek hatására a kiválasztott lap a játéktér közepén jelenik meg.
- **Pontozás és győzelem:** A felhasználói felület valós időben nyomon követi és megjeleníti az aktuális kártyák pontértékeit, valamint az összesített meccs-pontokat.
- **Lezárás:** A játék célja a 7 pont elérése. Amint valamelyik fél megszerzi a győzelemhez szükséges pontmennyiséget, a szoftver ezt egyértelműen jelzi a résztvevőknek, majd mindkét játékost automatikusan visszairányítja a főmenübe.

2.3. Oldaltérkép és navigáció

A webes Snapszer alkalmazás felépítése a letisztult és gyors használhatóság elvét követi. Annak érdekében, hogy a játékosok a lehető leggyorsabban játékba kerüljen, a felületek közötti navigáció egy logikus, jól áttekinthető folyamatra épül.

A 2.1. ábra az alkalmazás főbb nézeteit és az azok közötti átmeneteket (felhasználói útvonalat) szemlélteti.

2.3.1. Oldaltérkép



2.1. ábra. Oldaltérkép

Az Oldaltérkép nevű diagram elkészítéséhez a draw.io [1] nevű alkalmazást használtam.

2.3.2. Navigáció

A játékelmény szempontjából a navigáció az alábbi főbb állomásokból áll össze:

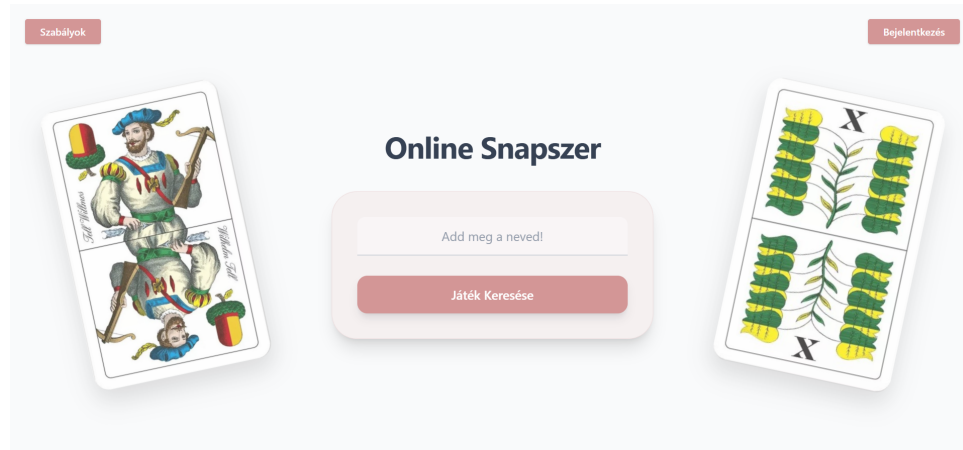
- **Főoldal:** Az alkalmazás központi csomópontja.
- **Szabály oldal:** A játékszabályokat bemutató információs felület.
- **Bejelentkezés:** A felhasználói hitelesítés helyszíne.
- **Regisztráció:** Új fiók létrehozására szolgáló űrlap.
- **Játék oldal:** A tényleges játékmenet asztala.
- **Statisztika:** A bejelentkezett felhasználók személyes eredményeinek összesítője.

A következőkben az egyes oldalak részletes szerepét és felépítését mutatom be.

Főoldal

A Főoldal az alkalmazás központi csomópontja, ahonnan a rendszer összes többi funkciója közvetlenül vagy közvetve elérhető. Itt található a játék indításához és az

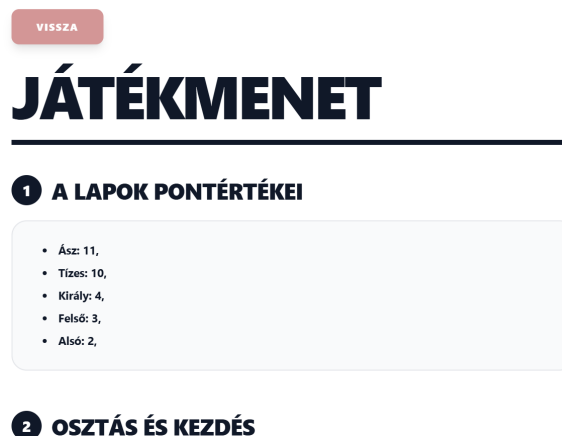
ellenfélkereséshez szükséges felület, valamint innen navigálhatunk a szabályzat, illetve a bejelentkezés irányába is. Sikeres azonosítás után a Főoldalról válik elérhetővé a személyes statisztikák megtekintése is.



2.2. ábra. Főoldal

Szabály oldal

Ezen az aloldalon a Snapszer kártyajáték részletes szabályzata olvasható. A Főoldalról közvetlenül elérhető felület célja, hogy a kezdő játékosok elsajátíthassák az alapokat, a tapasztaltabbak pedig felfrissíthessék tudásukat a lapok értékeiről és a speciális bemondásokról (például a húsz és negyven).

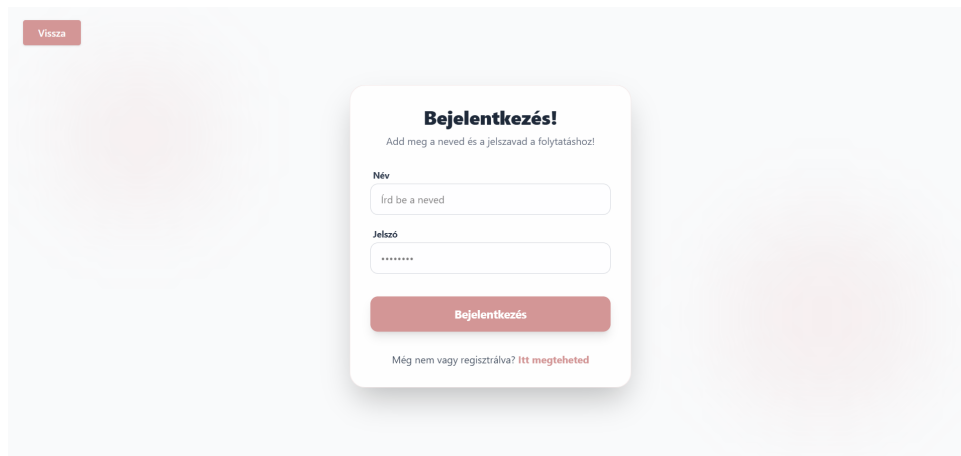


2.3. ábra. Szabály oldal

Bejelentkezés

A Bejelentkezés oldal szolgál a már regisztrált felhasználók hitelesítésére és profiljuk elérésére. A Főoldalról nyíló nézeten a megfelelő azonosítók (felhasználónév

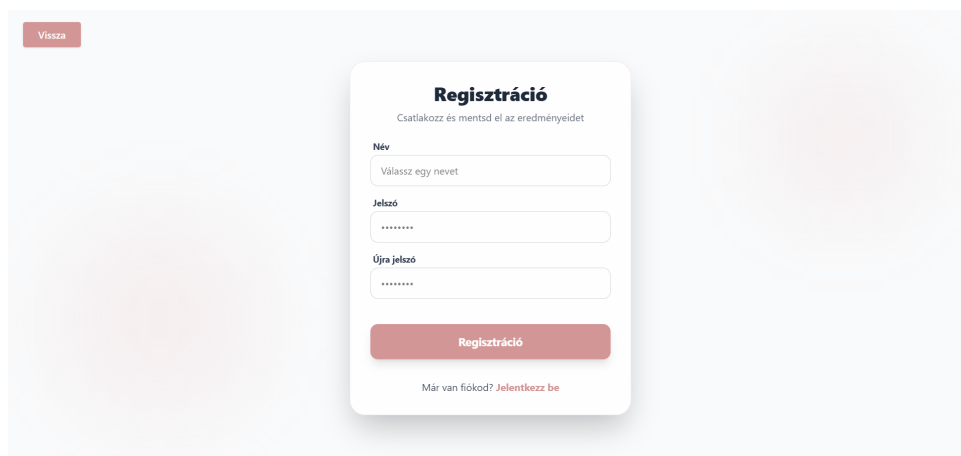
és jelszó) megadásával a játékos beléphet a fiókjába. Ez a felület funkcionál egyben belépési pontként a regisztrációs oldal felé is, a még fiókkal nem rendelkező látogatók számára.

The image shows a login form titled "Bejelentkezés!". At the top left is a "Vissza" button. The form itself is a white card with a red border. It contains the title "Bejelentkezés!" in bold, followed by the instruction "Add meg a neved és a jelszavad a folytatáshoz!". There are two input fields: "Név" (Name) with the placeholder "Írd be a neved" and "Jelszó" (Password) with a masked input "*****". Below these is a red "Bejelentkezés" button. At the bottom, there is a link: "Még nem vagy regisztrálva? [Itt megteheted](#)".

2.4. ábra. Bejelentkezés oldal

Regisztráció

Ahogy az oldaltérképen is látható, a Regisztráció oldal szorosan kapcsolódik a Bejelentkezéshez, és kizárólag onnan érhető el. Ezen a felületen egy letisztult űrlap kitöltésével a felhasználók létrehozhatják saját fiókjukat, amely lehetővé teszi a későbbi játékeredmények és statisztikák elmentését.

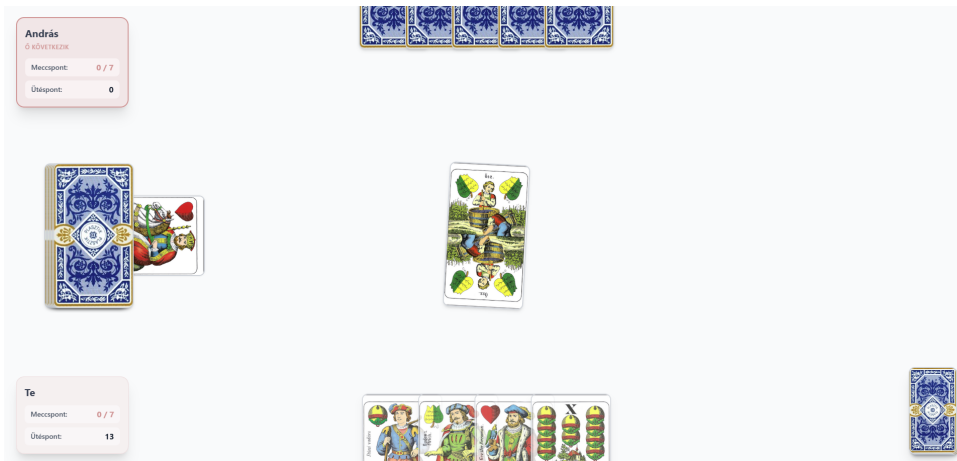
The image shows a registration form titled "Regisztráció". At the top left is a "Vissza" button. The form is a white card with a red border. It contains the title "Regisztráció" in bold, followed by the instruction "Csatlakozz és mentsd el az eredményeidet". There are three input fields: "Név" (Name) with the placeholder "Válassz egy nevet", "Jelszó" (Password) with a masked input "*****", and "Újra jelszó" (Repeat password) with a masked input "*****". Below these is a red "Regisztráció" button. At the bottom, there is a link: "Már van fiókod? [Jelentkezz be](#)".

2.5. ábra. Regisztrációs oldal

Játék oldal

A Játék oldal a mérkőzések tényleges helyszíne, maga a virtuális kártyaasztal. Ide a játékosok a Főoldalról induló sikeres meccskeresési folyamat után, emberi beavat-

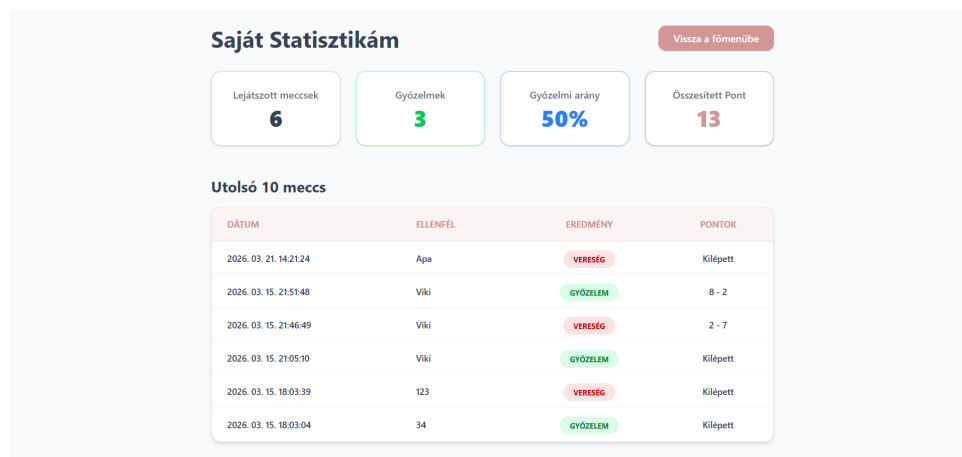
kozás nélkül, automatikusan kerülnek át. A felületen történik a kártyák interaktív kijátszása, az aktuális pontok valós idejű követése, valamint a győzelmi feltételek teljesülése esetén a játszma lezárása.



2.6. ábra. Játék oldal

Statisztika

A Statisztika egy extra funkciót kínáló felület, amely kizárólag a sikeresen bejelentkezett felhasználók számára válik elérhetővé a Főoldalról. Ezen a nézeten a játékosok nyomon követhetik a korábbi mérkőzéseik eredményeit, a lejátszott játszmák számát, valamint a győzelmek és vereségek arányát, ezáltal folyamatos visszajelzést kapva a saját teljesítményükről.



2.7. ábra. Statisztika oldal

2.3.3. Grafikai elemek és erőforrások

A webalkalmazás vizuális megjelenésének egyik legfontosabb eleme a játéktér és maguk a kártyalapok. A fejlesztés során cél volt a klasszikus, jól ismert kártyajáték hangulatának visszaadása a böngészőben is.

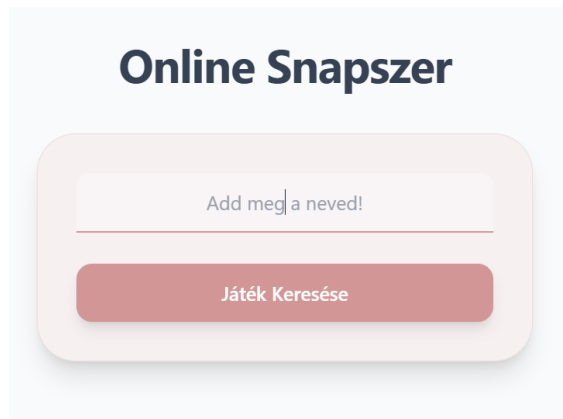
Ehhez a hagyományos magyar kártya digitalizált változatára volt szükség. A mérkőzések során megjelenő, a kártyákat ábrázoló képfájlokat külső forrásból [2] szereztem be, majd azokat integráltam a főoldal (két dísz kép) és a játék oldalba.

2.4. A játék indítása

A Snapszer webes alkalmazás úgy lett kialakítva, hogy a játékosok a lehető leggyorsabban, bonyolult beállítások nélkül játékba kerülhessenek. A játék indítása egy automatikus ellenfélkereső rendszeren keresztül történik.

2.4.1. Név megadása

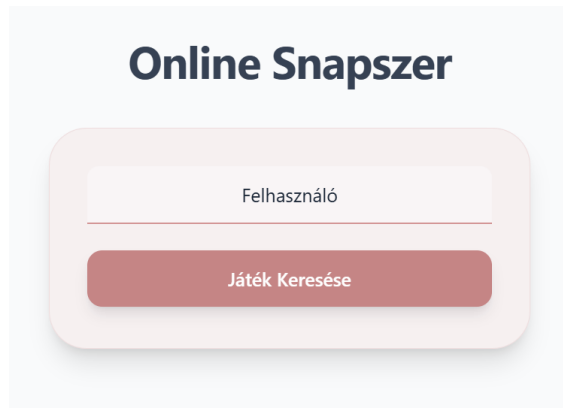
A felület megnyitása után első lépésként egy tetszőleges becenév (felhasználónév) megadása szükséges, amely a játék során azonosít téged. Regisztráció vagy fiók létrehozása nem kötelező.

A képen az "Online Snapszer" című webalkalmazás név megadási felülete látható. A felület fehér háttérrel rendelkezik, és a központi részen egy világos rózsaszínű, lekerekített sávban helyezkedik el a beviteli mező. A sáv tetején a "Online Snapszer" cím szerepel. A beviteli mezőben a "Add meg a neved!" felirat látható. A mező alatt egy piros gomb található, amelyen a "Játék Keresése" felirat van.

2.8. ábra. Név megadása

2.4.2. Játék keresése

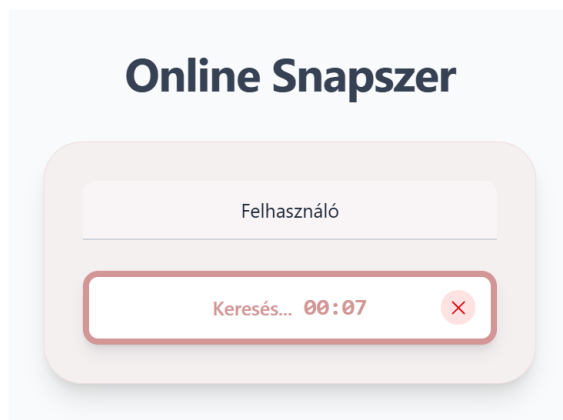
A főmenübe jutva a játék indításához csupán a "Játék indítása" (vagy Keresés) gombra kell kattintanod, miután megadtad a kívánt nevedet.



2.9. ábra. Játék keresés elindítása

2.4.3. Várakozás és kezdés

Miután megadtunk a nevünket és rákattintottunk a **"Játék Keresése"** gombra, elindul az ellenfélkeresés, és a képernyőn megjelenik egy visszaszámláló. A rendszer ekkor a háttérben egy másik, szintén várakozó játékoskal próbál összepárosítani. Amint a rendszer talál egy szabad ellenfelet, a keresési képernyő eltűnik, és a játék azonnal, automatikusan kezdetét veszi az első kártyák kiosztásával.



2.10. ábra. Játék keresés

Játék keresés közben meglehet állítani a keresést az "X" gombbal.

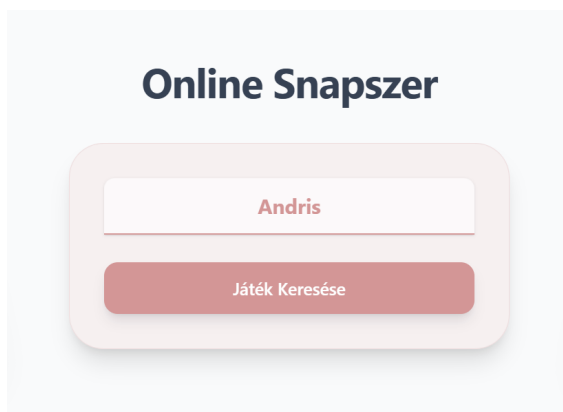
2.4.4. Játékkeresés bejelentkezett felhasználóként

Amennyiben a játékos rendelkezik regisztrált felhasználói fiókkal és be van jelentkezve, a játék indításának folyamata még gördülékenyebb.

Ebben az esetben a rendszer automatikusan a profilhoz tartozó nevet használja, így a név megadása lépés teljesen kimarad. A regisztrált nevünk az eddigi megszokott

szövegdobozban jelenik meg. A főmenüben elegendő a **"Játék Keresése"** gombra kattintani, és azonnal kezdetét veszi az ellenfélkeresés.

Fontos különbség a vendég módhoz képest, hogy a bejelentkezett állapotban lejátszott mérkőzések eredményeit (győzelmek, vereségek) a rendszer a háttérben rögzíti. Ezek az adatok elmentésre kerülnek a felhasználó személyes statisztikájába, és lehetőséget biztosítanak a globális ranglistán való szereplésre.



2.11. ábra. Bejelentkezett keresés

2.5. Regisztráció és bejelentkezés

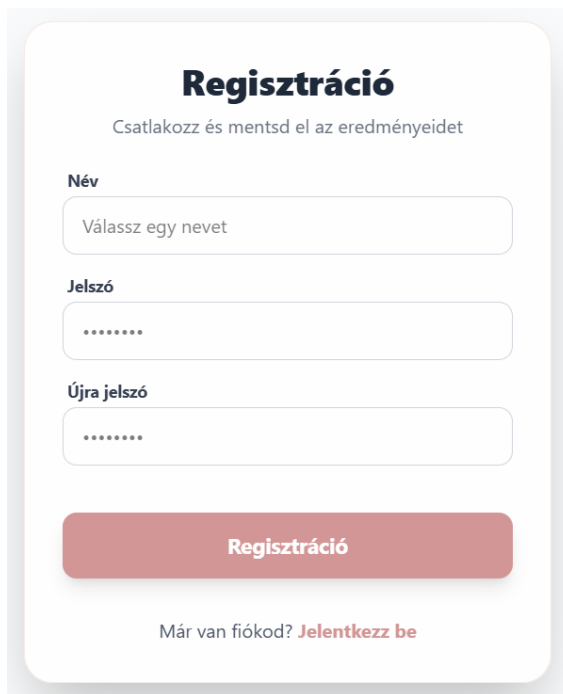
A Snapszer webalkalmazás lehetőséget biztosít arra, hogy vendégként, fiók létrehozása nélkül is azonnal asztalhoz ülhess. Azonban a teljes játékelmény és az elért eredményeid megőrzése érdekében érdemes saját felhasználói fiókot regisztrálnod.

2.5.1. Fiók létrehozása (Regisztráció)

A regisztrációs felületet a Főoldalról vagy a Bejelentkezés oldalról tudod elérni. Az űrlap kitöltésekor mindössze egy felhasználónevet és egy jelszót kell megadnod az alábbi szabályok szerint:

- **Felhasználónév:** A nevednek legalább 3, de legfeljebb 20 karakterből kell állnia. Ez a szabály biztosítja, hogy a játékosnévek könnyen azonosítani tudjanak, és a neved tökéletesen, levágás nélkül jelenjen meg a kártyaasztalon.
- **Jelszó:** A fiókad védelme érdekében a jelszónak minimum 6 karakter hosszúságúnak kell lennie.

Adatvédelmi megjegyzés: A rendszer a jelszavakat szigorúan titkosítva tárolja. Ez azt jelenti, hogy a tényleges jelszavadhoz senki még az alkalmazás készítője sem férhet hozzá, így a profilod maximális biztonságban van.



A regisztrációs űrlap a következőképpen néz ki:

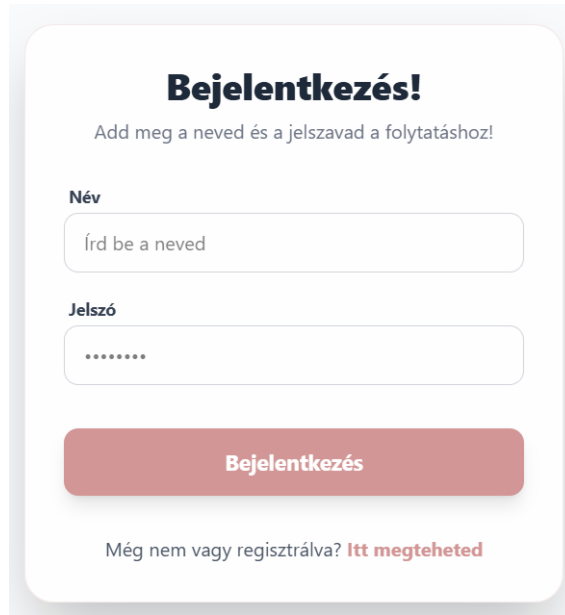
- Regisztráció** (főcím)
- Csatlakozz és mentsd el az eredményeidet (aláírás)
- Név** (címkézés)
- Válassz egy nevet (bemenet)
- Jelszó** (címkézés)
- (bemenet)
- Újra jelszó** (címkézés)
- (bemenet)
- Regisztráció** (gomb)
- Már van fiókod? [Jelentkezz be](#) (link)

2.12. ábra. A regisztrációs űrlap

2.5.2. Bejelentkezés és annak előnyei

Már meglévő fiók esetén a Bejelentkezés oldalon tudsz belépni a profilodba a felhasználóneved és jelszavad megadásával. A bejelentkezett állapot a következő kényelmi funkciókat és extra lehetőségeket nyitja meg számodra:

- **Személyes statisztikák:** Ez a regisztráció legfőbb előnye. A rendszer minden bejelentkezve lejátszott mérkőzésed eredményét elmenti. A Főoldalról elérhető *Statisztika* nézetben bármikor visszanézheted, hány meccset játszottál összesen, ebből mennyit nyertél és veszítettél, így nyomon követheted a saját fejlődésedet.
- **Gyorsabb játékkeresés:** Bejelentkezés után a rendszer megjegyzi a profilodat. Amikor a "Játék Keresése" gombra kattintasz, már nem kell minden alkalommal újra beírnod a becenevedet; a rendszer automatikusan a fiókod nevét használja, és a keresés azonnal, egyetlen kattintással elindul.

A bejelentkezési felület egy egyszerű, modern dizájnban. A címfejlécen a "Bejelentkezés!" felirat szerepel. Alatta egy utasítás olvasható: "Add meg a neved és a jelszavad a folytatáshoz!". Két inputmező található: az első a "Név" címmel, a második a "Jelszó" címmel. A jelszómezőben a karakterek pontokkal vannak elrejtve. A mezők alatt egy piros "Bejelentkezés" gomb található. A gomb alatt egy link van: "Még nem vagy regisztrálva? [Itt megteheted](#)".

Bejelentkezés!

Add meg a neved és a jelszavad a folytatáshoz!

Név

Írd be a neved

Jelszó

.....

Bejelentkezés

Még nem vagy regisztrálva? [Itt megteheted](#)

2.13. ábra. A bejelentkezési felület

2.6. Játékmenet

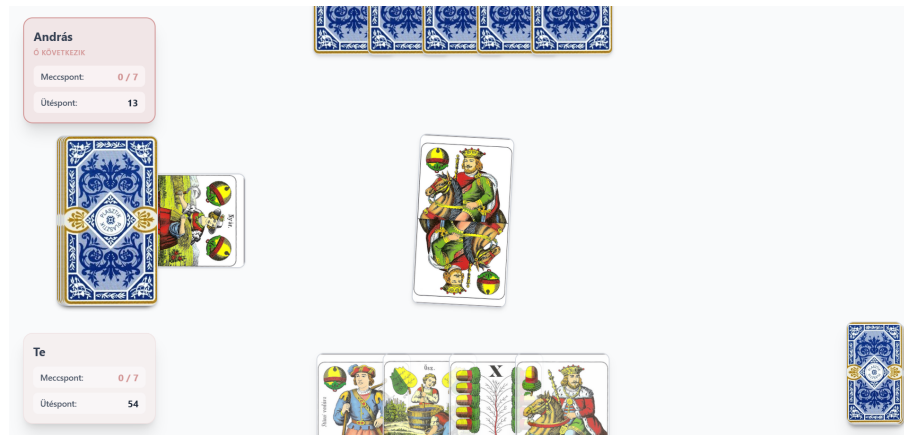
Ebben a fejezetben a mérkőzések során tapasztalható tényleges játékmenetet és a virtuális kártyaasztal funkcióit mutatom be. Részletesen végigvesszük a kártyák kijátszásának menetét, az aktuális pontok folyamatos megjelenítését, a szabálytalan (illegális) lépések vizuális és szöveges jelzését, valamint az olyan speciális Snapszer-szabályok felületi kezelését, mint a 20 és 40 bemondása, illetve a pakli takarása.

2.6.1. Kártyakijátszás és szabályosság vizsgálata

A játéktéren a saját lapjaink a képernyő alsó részén helyezkednek el. Amikor a mi körünk következik, egy kártya kijátszásához elegendő csupán bal egérgommbal rákattintani a kiválasztott lapra. A rendszer a háttérben minden lépésnél automatikusan ellenőrzi a játék aktuális állását és a Snapszer szabályait.

Sikeres kártyakijátszás

Amennyiben a választott lépés szabályos, a kattintás után a kártya azonnal az asztal közepére kerül. Ezt követően a lépésjog automatikusan átkerül az ellenfélhez.



2.14. ábra. Szabályos lépés: a kártya az asztal közepére kerül

Illegális lépés jelzése

Ha olyan kártyát próbálunk meg lerakni, amely az adott szituációban nem szabályos (mert a pakli elfogyása vagy takarása után megsértjük a színkényszert vagy a felülütési kényszert), a rendszer megakadályozza a lap kijátszását.

Ilyenkor az oldal egyértelmű vizuális visszajelzést ad: a kiválasztott kártya egy vastag piros keretet kap, és egy rövid, figyelmeztető rezgő animációt játszik le. Ezzel egy időben a képernyő felső részén megjelenik egy felugró ablak (popup), amely szövegesen is tájékoztat a hiba okáról (például figyelmeztet, hogy kötelező az ütés- vagy a színkényszer betartása).



2.15. ábra. Szabálytalan lépés vizuális jelzése és a felugró hibaüzenet

2.6.2. Játékosállapot és pontszámok kijelzése

A játéktér bal felső és alsó részén folyamatosan látható mindkét játékoshoz (a felhasználóhoz és az ellenfélhez) tartozó információs panel. Ezek a kis ablakok nyúj-

tanak kulcsfontosságú visszajelzést a mérkőzés aktuális állásáról és a játékosok státuszáról.

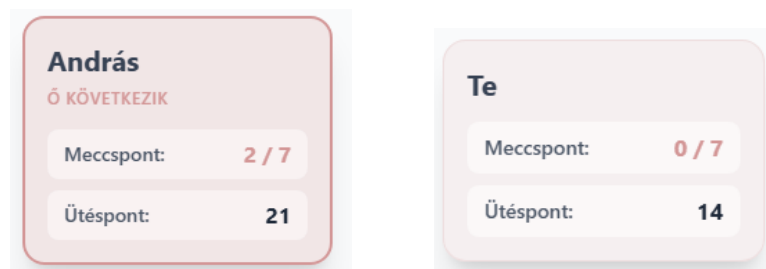
Ezek a panelek az alábbi információkat jelenítik meg:

- **Játékos neve:** Az azonosításhoz használt becenév (felhasználó esetében "Te").
- **Lépés státusz:** Szöveges jelzés arról, hogy az adott játékos következik-e.
- **Meccspont:** A mérkőzés megnyeréséhez szükséges összesített pontszám.
- **Ütéspon:** Az aktuális körben (partiban) a játékos által elvitt kártyák összesített értéke. Cél a 66 pont elérése.

Példa a lépésjog jelzésére és a vizuális kiemelésre

A rendszer vizuális visszajelzést ad arról, hogy éppen ki következik. A lépésjog egyértelmű jelzése érdekében az éppen soron lévő játékos információs ablaka kap egy színezett keretet.

Tekintsük a 2.16. ábrát példaként egy olyan játékhelyzetre, ahol az ellenfél következik:



2.16. ábra. Információs panelek: (balra) az ellenfél következik, (jobbra) a saját állapotunk várakozás közben

Ebben a szituációban az ellenfelünk, "András" következik. Ahogy a bal oldali ábrán látható:

- Az ő panelje egy vastagabb, színezett keretet kapott.
- A neve alatt megjelenik az "Ő KÖVETKEZIK" szöveges jelzés.
- A panelről leolvasható, hogy 2 meccspontja és 21 ütéspontja van.

A jobb oldali ábra a mi saját állapotunkat ("Te") mutatja ebben a pillanatban. Mivel nem mi következünk:

- A mi panelünk körül nincs kiemelő keret.
- Nincs szöveges jelzés a név alatt.
- Látható, hogy jelenleg 0 meccspontunk és 14 ütéspontunk van.

Amint András lerakja a kártyáját, a kiemelés azonnal eltűnik az ő ablakáról, és a keret meg fog jelenni a mi panelünk körül, jelezve, hogy ránk szállt a lépésjog.

2.6.3. Saját ütések megtekintése

A Snapszer kártyajátékban a stratégia egyik kulcseleme a már kijátszott lapok és a saját ütések ismerete. A webalkalmazás egy interaktív felületi elemmel segíti a játékost abban, hogy bármikor gyorsan ellenőrizhesse a már megszerzett lapjait.

A funkció a játéktér jobb alsó sarkában található. A felület alaphelyzetben helytakarékosági okokból csak egy lefordított kártyapaklit mutat, a részletes lista pedig kizárólag interakcióra jelenik meg:

- **Dinamikus megjelenítés:** Az "ÜTÉSEID" feliratú információs ablak csak akkor válik láthatóvá, amikor a felhasználó a kurzort a pakli fölé viszi.
- **Vizuális tartalom:** Ebben a felugró panelben megjelenik az összes olyan kártya, amelyet a játékos az adott körben eddig megszerzett. Ez segíti hátralévő lapok kikövetkeztetését.



2.17. ábra. Az ütések részletes megjelenítése az egérkurzor rávitelére

2.7. Speciális lépések és stratégiai lehetőségek

A Snapszer kártyajáték izgalmát és taktikai mélységét a hagyományos ütéseken túlmutató speciális lépések adják. A webalkalmazás ezeket a lehetőségeket intuitív módon és vizuális segítséggel támogatja.

2.7.1. Húsz és negyven bemondása

A játék során extra pontokat szerezhethünk, ha azonos színű Király és Felső párral rendelkezünk. A szoftver ezt a folyamatot teljesen automatizálta:

- **Aktiválás:** Amint a játékos kijátssza a nála lévő páros egyik tagját (Királyt vagy Felsőt), és a kezében van a pár másik fele, a rendszer ezt azonnal felismeri.
- **Vizuális visszajelzés:** A bemondás tényét egy közepén felül megjelenő felugró ablak jelzi, amely mutatja a szerzett pontértéket (aduszín esetén 40, egyébként 20 pont).
- **Pontozás:** A pontok automatikusan hozzáadódnak a játékos ütésponthoz, és a pontszámkijelző azonnal frissül.



2.18. ábra. A 20/40 bemondást jelző felugró ablak

2.7.2. Aducsere

Ha a játékosnál van az aduszín legalacsonyabb lapja (az Alsó), lehetősége van azt kicserélni az asztal közepén felütött adukártyára.

- **Szabályos csere:** Az aducserére csak akkor van lehetőség, ha nálunk van az adu alsó, és éppen mi következünk (lépésjogunk van). A végrehajtáshoz egyszerűen rá kell kattintani az asztalon fekvő, félig a pakli alatt lévő adukártyára. Ekkor a két lap helyet cserél.

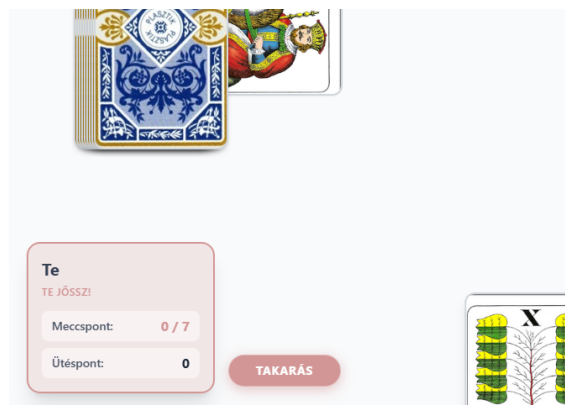
- **Szabálytalan csere jelzése:** Amennyiben a játékos akkor kísérel meg adut cserélni, amikor a feltételek nem teljesülnek (például nincs a kezében az adu alsó). A rendszer hibaüzenettel akadályozza meg a műveletet.

Ilyenkor az asztalon fekvő adukártya egy vastag piros keretet kap, és egy rövid, figyelmeztető rezgő animációt játszik le. Ez a vizuális visszajelzés egyértelműen jelzi a felhasználónak, hogy a csere az adott pillanatban nem hajtható végre.

2.7.3. Takarás (A pakli lezárása)

A takarás a legkockázatosabb, de egyben a legtöbb pontot hozó stratégiai elem. Ezzel jelezzük, hogy a kezünkben lévő lapokból további húzás nélkül is képesek vagyunk megnyerni a kört.

- **Elérhetőség:** A "Takarás" gomb csak akkor jelenik meg a felületen, ha mi vagyunk soron, és még legalább 3 lefordított lap található a talonban (a húzó-pakliban).
- **Hatás:** A gombra kattintva a pakli "lezárul". Ezután nincs több lapfelvétel, és a játékosoknak azonnal tartaniuk kell a szín és ütéskenyszert.
- **Elhelyezkedés:** A gomb a felhasználó saját állapotjelző kisablaka (státusz panel) mellett jelenik meg.



2.19. ábra. A Takarás gomb megjelenése a játéktéren

A takarás vizuális visszajelzései

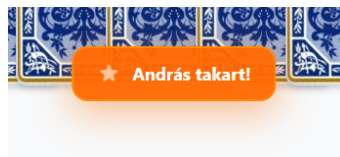
Miután a játékos a gombra kattintva lezárja a paklit, a rendszer két egyértelmű vizuális jelzéssel tudatja a résztvevőkkel, hogy a játékmenet megváltozott, és érvénybe lépett a szín és ütéskenyszer.

- **A talon lezárása:** A takarás pillanatában a korábban felütött adukártya rákerül a talon (húzópakli) tetejére.



2.20. ábra. A talon lezárása: az adukártya a pakli tetejére kerül

- **Megerősítő ablak:** A művelet végrehajtásakor a képernyő közepén felül egy rövid ideig látható felugró ablak is megjelenik, amely szövegesen is jelzi a takarás tényét mindkét játékos számára.



2.21. ábra. A takarást megerősítő felugró ablak

3. fejezet

Fejlesztői dokumentáció

Ebben a fejezetben az alkalmazás technikai megvalósításának részleteit mutatom be.

3.0.1. Célközönség és a szoftver célkitűzései

Az alkalmazás elsődleges célja, hogy modern, digitális alternatívát nyújtson a snapszer kártyajátéknak. Lényegében azoknak készült, akik szeretnének akkor is játszani, amikor nincsenek közvetlenül körülöttük azok a személyek (például családtagok vagy barátok), akikkel hagyományosan, fizikai kártyapaklival szoktak játszani. Az alkalmazás áthidalja a fizikai távolságot, és lehetőséget ad a közös kikapcsolódásra akkor is, ha a felek nincsenek egy helyiségben. A fejlesztés során négy fő célcsoportot határoztam meg, akiknek az alkalmazás funkcionalitását és felületét dedikáltam:

- **A klasszikus kártyajátékok kedvelői:** Azok a felhasználók, akik ismerik és szeretik a snapszer szabályait, de a mindennapok során nehézséget okoz számukra fizikai partnert találni a játékhoz. A snapszer.com élesített rendszere nekik biztosít azonnali párosítási lehetőséget a várólista segítségével.
- **Digitális platformot kereső közösségek:** Az alkalmazás célközönségébe tartoznak azok a baráti körök vagy ismerősök, akik földrajzi távolságtól függetlenül szeretnének egymás ellen játszani. A rendszer regisztrációs és statisztikai modulja lehetővé teszi számukra a hosszú távú versengést és az eredmények visszakövetését.

- **Alacsony belépési küszöböt igénylő felhasználók:** Mivel az applikáció böngésző alapú és nem igényel telepítést, célközönségnek tekinthetők a kevésbé technikai beállítottságú játékosok is.
- **A játékot tanulni vágyók (kezdők):** A felület ideális azoknak is, akik most szeretnék megtanulni a snapszer alapjait. Az alkalmazás rendelkezik szabályzat oldallal, amely segít az elméleti tudás elsajátításában, a szoftverbe épített automatizált szabályellenőrzés pedig a gyakorlatban tanítja meg az érvényes lépéseket, megakadályozva a szabálytalan kártyarakást.

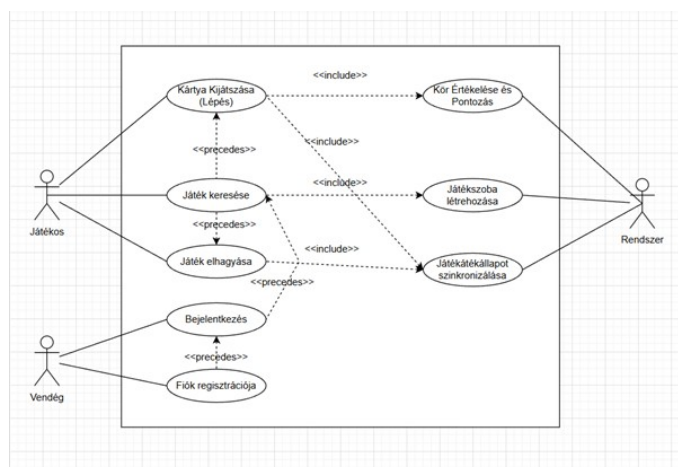
A szoftver végső célja egy olyan közösségi tér létrehozása, ahol a hagyományőrzés és a modern technológia találkozik, biztosítva a játék tisztaságát az automatizált szabályellenőrzéssel és az igazságos pontszámítással.

3.1. Implementációt megelőző rendszertervek

Ez a szakasz azokat az előzetes terveket tartalmazza, amelyek a kódolás megkezdése előtt készültek, és referenciaként szolgáltak a fejlesztés során. Az ábrák a tervezési fázisból és az szoftver architektúrája fázisból készültek.

3.1.1. Használatieset-diagram (Use-case diagram)

Ez a diagram a fejlesztés megkezdése előtt rögzített funkcionális elvárásokat és a felhasználói szerepkörök interakcióit szemlélteti.



3.1. ábra. Előzetes használatieset-diagram
Az ábra a draw.io [1] alkalmazás segítségével készült.

A használatieset-diagram (Use-case diagram) funkcionális leírását az alábbi táblázat egészíti ki:

Given (Adott)	When (Amikor)	Then (Akkor)
Vendégként tartózkodom a regisztrációs oldalon.	Kitöltöm a szükséges adatokat (felhasználónév, érvényes e-mail cím, jelszó) és rákattintok a „Regisztráció” gombra.	A Rendszer létrehozza a fiókomat.
Vendégként tartózkodom a bejelentkezési oldalon.	Megadom a regisztrált felhasználóneveimet és jelszavam, és rákattintok a „Belépés” gombra.	A Rendszer hitelesíti az adataimat, és sikeresen a főmenübe irányít.
Játékosként be vagyok jelentkezve, és a főmenüben vagyok.	Rákattintok a játék keresése gombra.	A Rendszer létrehoz egy szobát és keres nekem ellenfelet.
Játékosként egy játékban vagyok, és én vagyok soron.	A kezemben lévő kártyák közül érvényesen kijátszok egyet.	A Rendszer elfogadja a lépést, a kijátszott kártya megjelenik az asztalon, és a sor a következő játékosra száll.
Játékosként van egy királyom és egy felsőm azonos színből és én vagyok soron.	Egy gombot kattintva kijátszom a 20/40-et és kiválasztom a királyt vagy a felsőt.	A Rendszer elfogadja a lépést, hozzárendeli a pontokat és megjeleníti a lapot az asztalon.
Két játékos kijátszotta a kártyáját az adott körben.	A Rendszer kiértékeli a két kijátszott kártyát a snapszer szabályai szerint.	A Rendszer megállapítja a kör nyertesét, kiosztja a pontokat, és frissíti a kijelzőt.
Véget ér egy kör.	A kört nyert játékosnak összegyűlik a 66 pont.	A rendszer kihirdeti a játékost, aki nyerte a partit.
Játékosként egy játékteremben tartózkodom, de még nem fejeződött be a parti.	Rákattintok a „Játék elhagyása” gombra.	A Rendszer eltávolít a játékteremből és átirányít a főmenübe.
Játékosként bejelentkezve a főmenüben tartózkodom.	Rákattintok a statisztikák gombra.	Megjelenik egy oldal, amin az eddig elért partikból láthatóak a statisztikák.

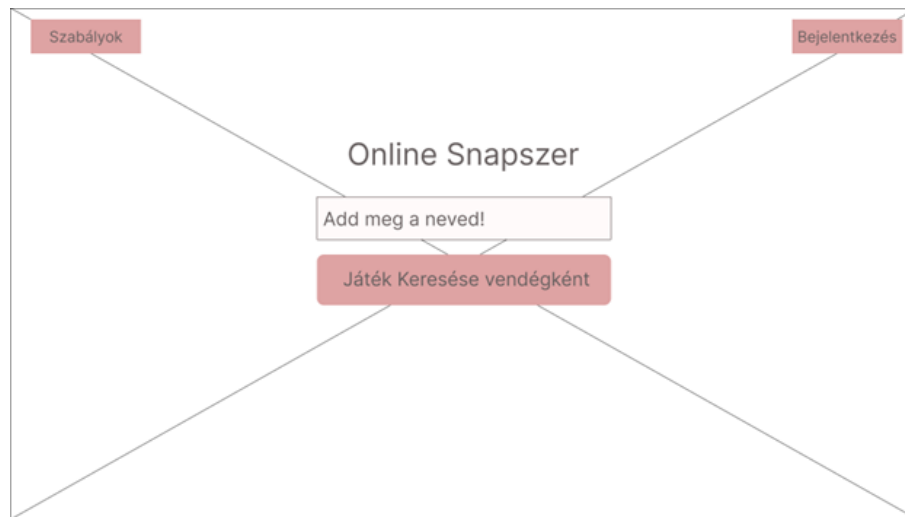
3.1. táblázat. A rendszer használati eseteinek részletezése

A táblázat a Microsoft Word [3] alkalmazás segítségével készült.

3.1.2. A felhasználói felület tervei (Figma)

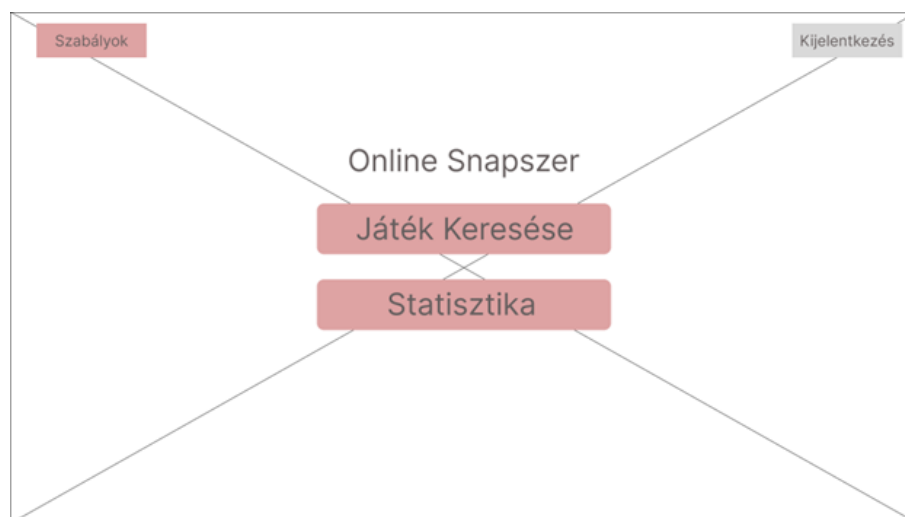
A vizuális tervezési szakaszban készült drótváztervek az alkalmazás ergonómiáját és a tervezett komponensek elrendezését rögzítették. Az összes drótváz ábrát a Figma [4] grafikai tervező szoftver segítségével készítettem el.

Bejelentkezés nélküli főoldal



3.2. ábra. Bejelentkezés nélküli főoldal drótváz ábrája

Bejelentkezett főoldal



3.3. ábra. Bejelentkezett főoldal drótváz ábrája

Bejelentkezés oldal



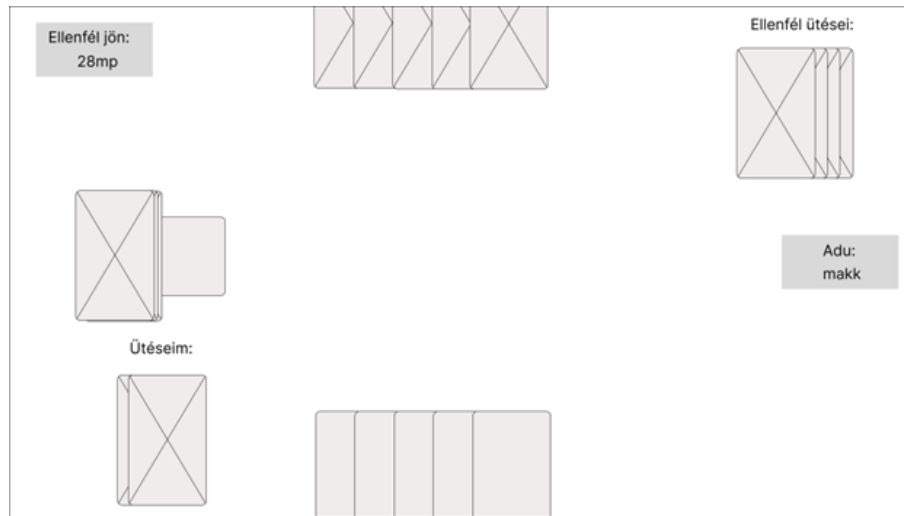
3.4. ábra. Bejelentkezés oldal drótváz ábrája

Regisztráció oldal

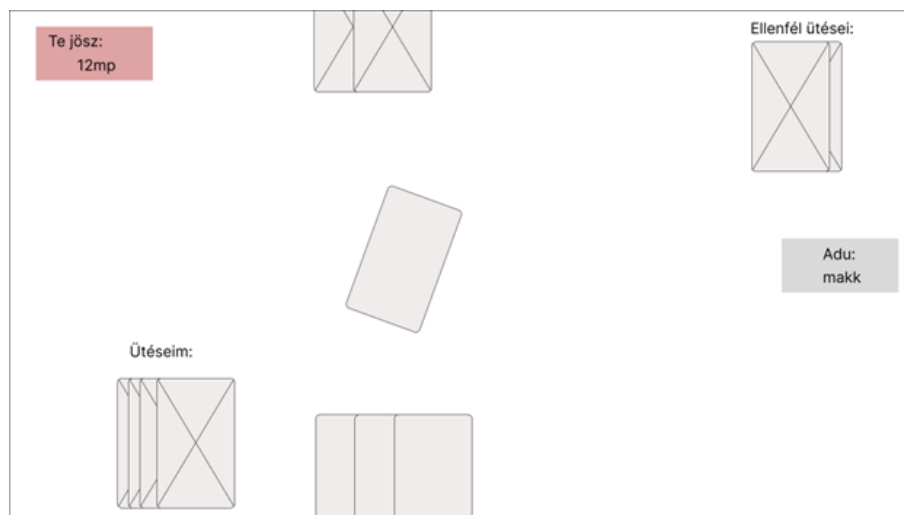


3.5. ábra. Regisztráció oldal drótváz ábrája

Játék oldal



3.6. ábra. Játékoldal drótváz ábrája, amikor az ellenfél van soron

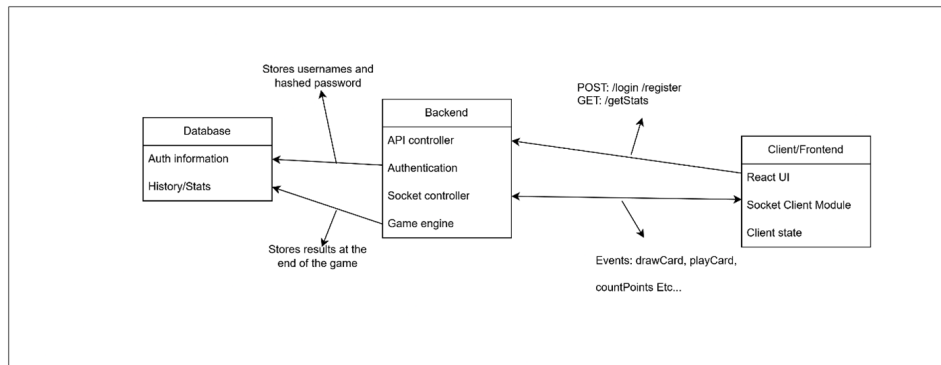


3.7. ábra. Játékoldal drótváz ábrája, amikor mi vagyunk soron

3.1.3. Komponens diagramok

Magas szintű architektúra

Az első diagram a tervezett felhasználói felületet (Client/Frontend), a kiszolgáló egységet (Backend) és az adatbázist (DB) és a kapcsolatait szemlélteti.

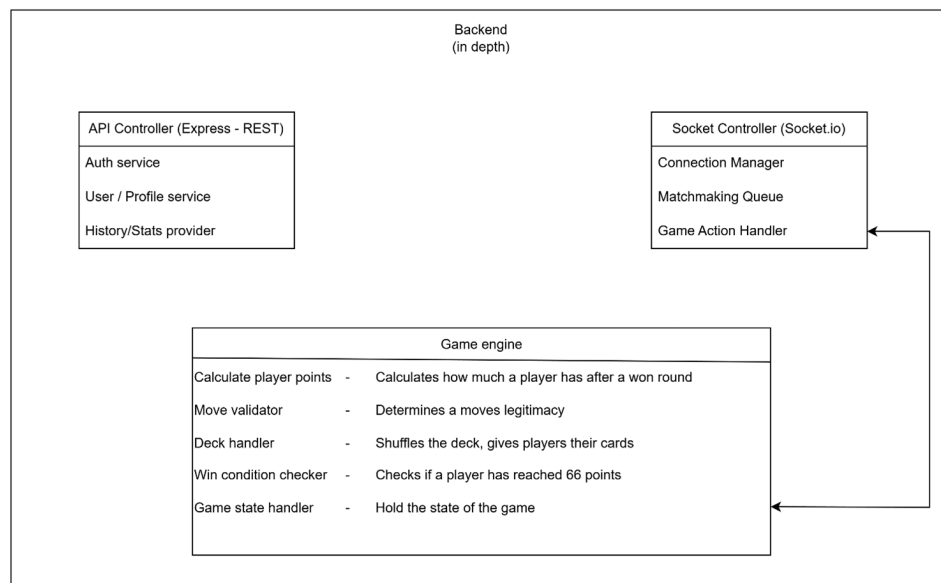


3.8. ábra. A rendszer három fő komponensének kapcsolata

Az ábra a draw.io [1] alkalmazás segítségével készült.

A Backend részletes felépítése

A második diagram részletes betekintést nyújt a tervezett Backend belső működésébe, elmagyarázva az API vezérlőt és a valós idejű kommunikációért felelős Socket modul feladatait.

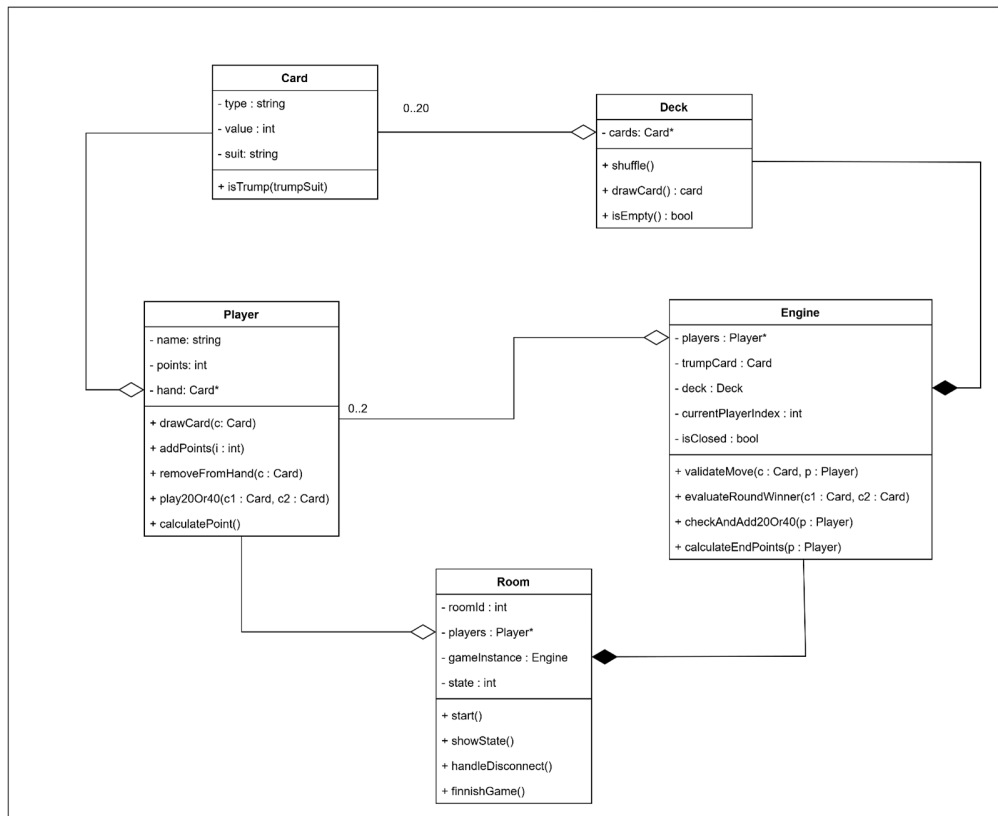


3.9. ábra. A Backend belső moduláris felépítésének terve

Az ábra a draw.io [1] alkalmazás segítségével készült.

3.1.4. Osztálydiagram (UML)

Az előzetes osztálydiagram a szoftver belső objektumorientált adatszerkezeteknek a tervét mutatja be.

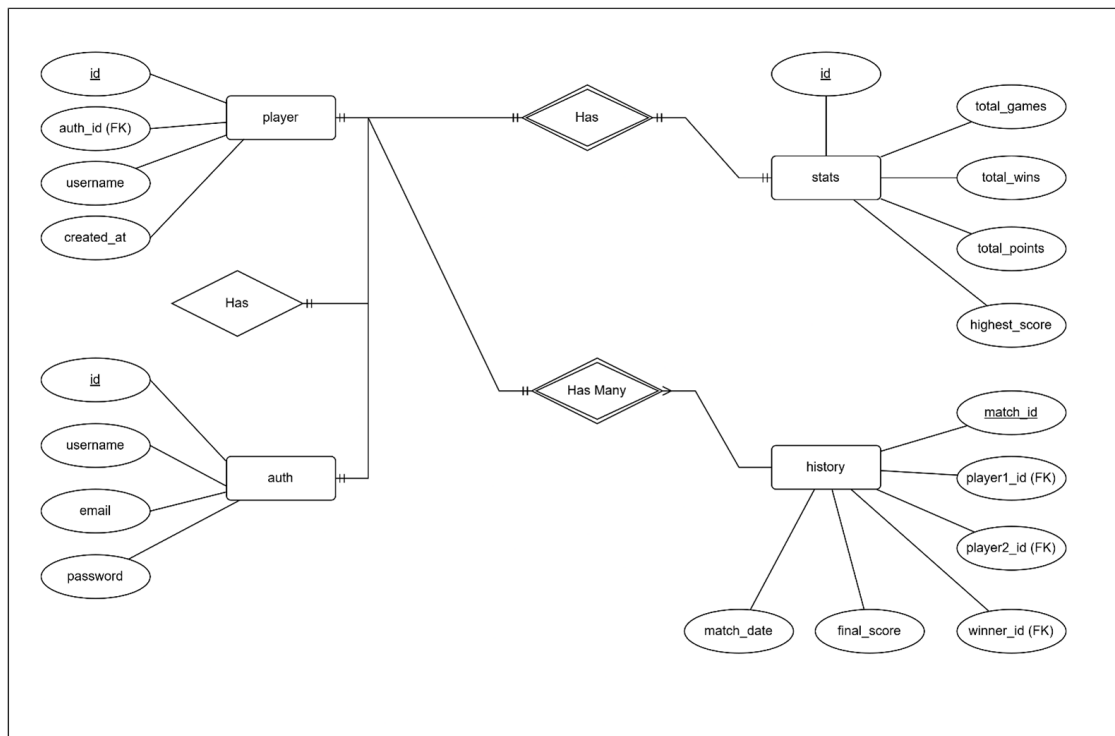


3.10. ábra. Tervezett osztálydiagram (UML)

Az ábra a draw.io [1] alkalmazás segítségével készült.

3.1.5. Egyed-kapcsolat diagram (ER diagram)

Az adatbázis-tervezés során készült ER diagram a tárolni kívánt adatok közötti logikai relációkat és a táblák tervezett sémáját ábrázolja.



3.11. ábra. Az adatbázis előzetes logikai terve (ER diagram)

Az ábra a draw.io [1] alkalmazás segítségével készült.

3.2. A szoftver belső struktúrája és adatábrázolása

A rendszer stabilitását és teljesítményét nagymértékben meghatározza az alkalmazott szoftverarchitektúra, ezért ebben a szakaszban a backend forráskódjának felépítését mutatom be. A kódolás során a modularitás elve érvényesült, így az egyes komponensek könnyen tesztelhetők, és a jövőbeli továbbfejlesztések alkalmával is akadálymentesen bővíthetők.

3.2.1. Moduláris felépítés és kódminőség

Jelentősen megnőtt a kód olvashatósága és karbantarthatósága, mivel szigorúan szét lettek választva a logikai rétegek a szerveroldalon. Különálló, objektumorientált osztályokba (például `Card.js`, `Deck.js`, `Player.js`) került a játékmenet alapvető logikája, így ezek a fájlok belső állapottal (encapsulation) rendelkeznek, és teljesen függetlenek a hálózati keretrendszertől. Kimagasló újrafelhasználhatóságot biztosít ez a megközelítés, ugyanis a központi játékmotor (`Engine.js`) lényegében bármilyen környezetben (akár egy lokális, asztali alkalmazásban is) futtatható lenne kód-

módosítás nélkül. Megkönnyítené egy mobilalkalmazás jövőbeli fejlesztését is ez az elszigetelt felépítés, mivel a játékmotor kódja könnyedén újrahasznosítható.

Kifejezetten biztonságos és robusztus a hitelesítési folyamat is, mivel az `authController.js` aszinkron vezérlőfunkciói felelnek a jelszavak hasheléséért és az adatok validálásáért. Garantált a szerveroldali adatintegritás, hiszen az adatbázisba történő írás során a kód a `sequelize.transaction` metódust alkalmazza, amely biztosítja, hogy szerverhiba esetén a félig végrehajtott módosítások automatikusan visszavonásra kerüljenek (rollback).

3.2.2. Adatábrázolási döntések (Sequelize ORM)

Kizárólag relációs adatbáziskezelőt alkalmaz a rendszer háttértára, amelynek a dinamikus eléréséhez és az adattípusok absztrakt definiálásához a Sequelize ORM szoftverkönyvtárat implementáltam[5]. JavaScript osztályokként modelleztem az entitásokat, hogy a fejlesztés során ne kelljen nyers SQL parancsokat írni, ezáltal védem az alkalmazást az SQL injection támadás ellen.

Hatékonyan valósulnak meg a táblák közötti összeköttetések, mert az `index.js` fájlban pontosan lettek definiálva a relációk, mint amilyen a `Player` és a `History` (mérkőzés-történet) modellek közötti egy-a-többhöz (1:N) kapcsolat is. Soha nem maradnak árva rekordok az adatbázisban a profilok törlése után, mivel a kapcsolatoknál minden esetben be lett állítva az `ON DELETE CASCADE` szabály, amely a szülő entitás eltávolításakor a kapcsolódó al-adatokat is megsemmisíti. Modern és skálázható megoldásnak számít az elsődleges kulcsok (Primary Key) ábrázolása is, ugyanis a hagyományos inkrementális sorszámozás helyett automatikusan generált, prediktálhatatlan `UUIDv4` azonosítókat (`DataTypes.UUID`) kapnak a rekordok az adatbázisban.

3.2.3. Kliensoldali architektúra és állapotkezelés

Az alkalmazás a teljes útvonalválasztást a `React Router` keretrendszerrel kezeli az `App.jsx` fájlban, oldalújrátöltések nélkül. A frontend kódja moduláris, hiszen a felhasználói felület önálló, funkcióspecifikus React komponensekre (például `GamePage`, `StatsPage`, `GuestPage`) bomlik.

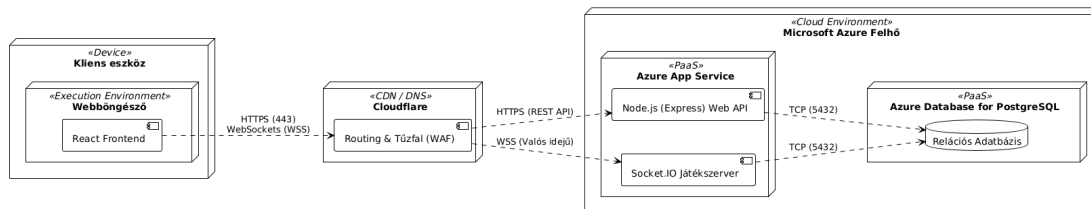
A valós idejű hálózati kommunikációt a `socket.js`-ben inicializált `Socket.IO` [6] kliens végzi, amelynek eseményeit a React beépített `useEffect` hookjai figyelik.

Különösen megbízható a vizuális visszacsatolás a játékmenet során, mert a szabálytalan lépéseket jelző animációkhoz és az értesítésekhez tartozó időzítéseket (timeouts) a `useRef` hookokkal izoláltan, a memóriában tartva kezeli a rendszer, elkerülve a memóriaszivárgást.

A JWT tokenek a böngésző `localStorage` tárolójába kerülnek, így magas szintű biztonságot és konzisztens felhasználói élményt garantál a munkamenet-kezelés. Szinkronban maradnak a megnyitott böngészőlapok, mivel a fő `App` komponens egy globális eseményfigyelővel monitorozza a tárhely változásait, így egy esetleges kijelentkezés azonnal és automatikusan érvényesül a felhasználó összes nyitott ablakában. A megjelenés reszponzív és modern az összes nézeten, erre a Tailwind CSS [7] keretrendszert alkalmaztam, amit közvetlenül a JSX struktúrába integráltam.

3.2.4. Fizikai komponens-telepítési diagram és infrastruktúra

Jól áttekinthető képet nyújt az alkalmazás architekturális felépítéséről a fizikai telepítési diagram. Az ábra megmutatja a Microsoft Azure felhőszolgáltatásokba kihelyezett szoftverkomponensek és a hálózati rétegek topológiáját.



3.12. ábra. A rendszer fizikai telepítési diagramja (Deployment Diagram)

A weboldal publikus elérése a Cloudflare [8] rendszerén keresztül valósul meg. A `https://snapszer.com` domain forgalmát ez a szolgáltató kezeli, és irányítja tovább a felhasználói kéréseket az Azure-ban hosztolt végpontok felé. A Cloudflare beépített hálózati tűzfala és automatikus DDoS-védelme proaktívan szűri a kártékony forgalmat, garantálva a rendszer biztonságát a külső kibertámadásokkal szemben.

A tényleges futtatókörnyezet teljes egészében a Microsoft Azure [9] felhőalapú ökoszisztémájára épül. Ez a modern, Platform as a Service (PaaS) megközelítés tehermentesíti a fejlesztőt a fizikai infrastruktúra fenntartásától. A frontend statikus lefordított fájljait, a REST API végpontokat és a valós idejű Socket.IO játékszerveret egyetlen közös, egyszerűen skálázható Azure App Service környezet szolgálja ki.

Az alkalmazás perzisztens adattárolásáért egy különálló, menedzselt Azure Database for PostgreSQL szerver felel. Ez az adatbázis rögzíti a felhasználói regisztrációkat, a profilok pontszámait és a mérkőzések részletes történetét. A kiszolgáló fizikailag független az App Service-től, de azonos virtuális hálózaton helyezkedik el, a belső adatbáziskommunikáció így gyors és zárt marad.

A komponensek közötti együttműködés és a hálózati adatáramlás jól szemléltethető egy konkrét játékesemény (például egy kártyalap kijátszásának) levezetésével:

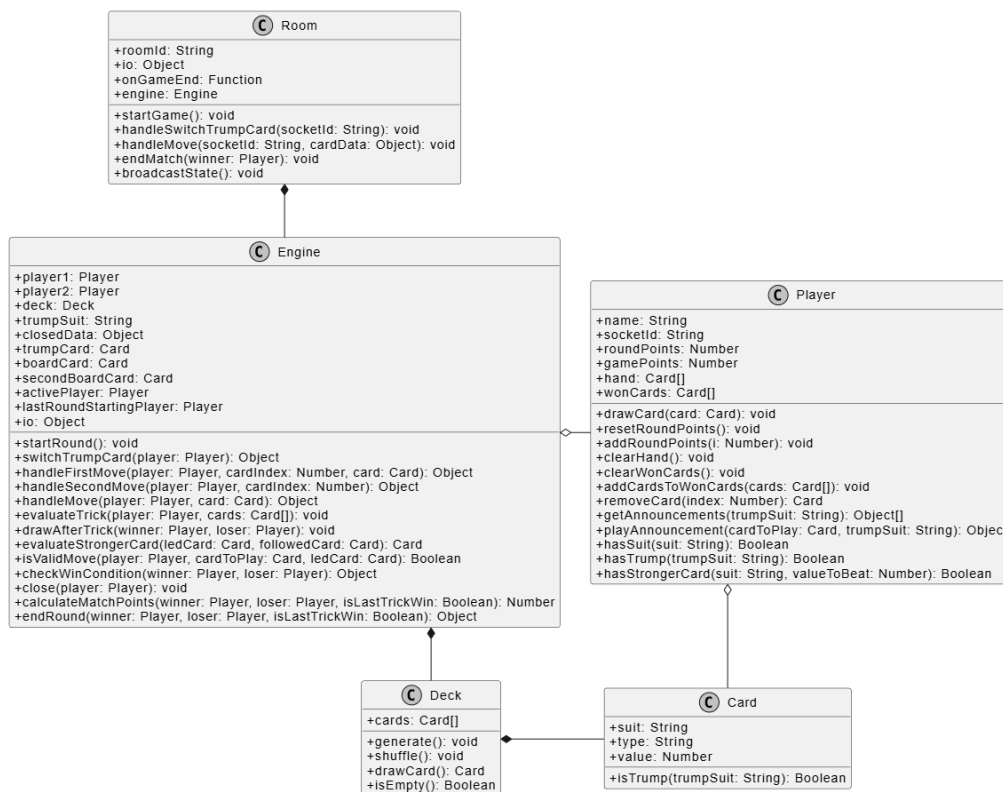
1. **Kliens interakció:** A játékos a böngészőben (React frontend) rákattint a megjátszani kívánt kártyára. A kliensoldali alkalmazás összeállít egy szabványos JSON adatcsomagot a lépés részleteivel.
2. **Hálózati továbbítás:** A böngésző egy titkosított, valós idejű WebSocket (WSS) csatornán keresztül elküldi a csomagot. A kérés beérkezik a Cloudflare hálózatába, amely szűri a forgalmat, majd továbbítja azt a felhőbe.
3. **Szerveroldali feldolgozás:** Az Azure App Service környezetben futó Node.js szerveren a Socket.IO kliens fogadja a kérést. Az adatokat átadja az aktív mérkőzést kezelő játékmotornak, amely validálja a lépést a snapszer szabályai szerint.
4. **Állapotszinkronizáció:** Érvényes lépés esetén a játékmotor módosítja a szerver memóriájában tárolt belső állapotot. A szerver ezután a `broadcastState` metódussal visszaküldi az új, frissített játékalapot mindkét résztvevőnek.
5. **Adatbázis tranzakció (opcionális):** Amennyiben a kijátszott kártyalap eldönti a teljes mérkőzést, a Node.js szerver egy biztonságos TCP kapcsolaton keresztül bejegyzi a végeredményt a PostgreSQL adatbázisba, véglegesítve a játékosok statisztikáit.

3.3. A megvalósult rendszerstruktúra

A fejlesztési folyamat során az előzetes tervek bizonyos pontokon változtak és alkalmazkodtak a technológiai kihívásokhoz, valamint a felmerülő új kihívásokhoz. Az alábbi diagramok a szoftver végleges, implementált állapotát tükrözik.

3.3.1. Végleges osztálydiagram (UML)

A megvalósult osztálydiagram bemutatja a tényleges az üzleti logikát kiszolgáló osztályokat, a köztük lévő tényleges kapcsolatokat. A diagram a végleges kódbázis alapján készült.



3.13. ábra. A rendszer megvalósult osztálydiagramja

Az ábra a PlantUML [10] eszköz segítségével készült.

Az osztálydiagramon szereplő komponensek a felelősségi körök egyértelmű szétválasztásának elve alapján lettek kialakítva. A rendszer főbb osztályai és azok funkciói a következők:

3.3.2. Osztályok

- **Card:** Az osztályok közül a legkisebb, amely egyetlen kártyalapot reprezentál. Tárolja a kártya színét, típusát és a játékban betöltött pontértékét. Képes meghatározni önmagáról, hogy az aktuális aduszín ismeretében ő maga adunak minősül-e.
- **Deck:** A kártyapakli állapotának és műveleteinek kezeléséért felelős osztály. Feladata a snapszer szabályainak megfelelő 20 lapos csomag generálása, meg-

keverése, valamint a kártyahúzás (drawCard) és a pakli ürülésének (isEmpty) logikai biztosítása. A keveréshez a Knuth keverési algoritmust alkalmaztam.

1. algoritmus Fisher–Yates (Knuth) keverés algoritmus

```

1: function SHUFFLE( $A$  tömb)
2:    $n \leftarrow \text{hossz}(A)$ 
3:   for  $i \leftarrow n - 1$  downto 1 do
4:      $j \leftarrow$  véletlen egész szám 0 és  $i$  között (inkluzív)
5:     Cseréljük meg az  $A[i]$  és  $A[j]$  elemeket
6:   end for
7: end function

```

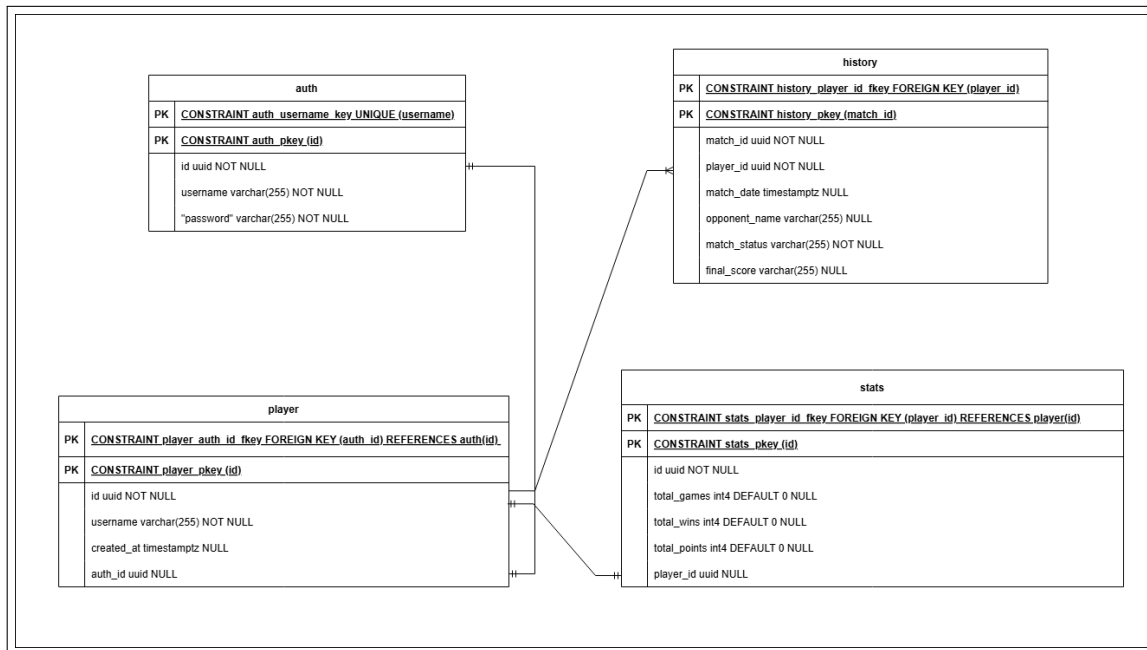
Az algoritmus forrása a Wikipédia[11].

- **Player:** Egy adott mérkőzésben résztvevő kliens szerveroldali reprezentációja. Nyilvántartja a játékos kezében lévő lapokat, a már megnyert (ütött) kártyákat, valamint az aktuális körben és a meccs során szerzett pontokat. Ez az osztály végzi a játékoshoz köthető specifikus ellenőrzéseket is, például a 20-as és 40-es mondások (announcements) felismerését és validálását.
- **Engine:** A játékszerver központi magja, amely a teljes üzleti logikát és a klasszikus Snapszer játékszabályokat implementálja. Független a hálózati rétegtől, kizárólag a játék menetéért felel. Feladata a körök elindítása és lezárása, az ütés- és színkényszer szigorú betartatása (isValidMove), a kártyalapok erősségének összehasonlítása, a pakli takarása (close), valamint a pontszámítás és a győzelmi feltételek ellenőrzése.
- **Room:** A hálózati kommunikáció és a játékmotor (Engine) közötti híd szerepét tölti be. Minden elindult mérkőzéshez létrejön egy szobapéldány, amely a Socket.IO eseményeket kezeli. Fogadja a kliensek lépéseit (kártyarakás, aducsere), továbbítja azokat validálásra a játékmotornak, majd a frissített játékállapotot (broadcastState) elküldi minden játékeseménynél a két kliensnek.

3.3.3. Végleges adatbázis séma (ER diagram)

Az alkalmazás adatperzisztenciáját biztosító adatbázis végleges szerkezete az alábbi egyed-kapcsolat diagramon (ER diagram) látható. Az ábra szemlélteti a lét-

rehozott táblákat, a tárolt adattípusokat, az elsődleges és idegen kulcsokat, valamint a köztük lévő relációkat.



3.14. ábra. A megvalósult adatbázis szerkezete (ER diagram)

Az ábra a draw.io [1] alkalmazás segítségével készült, a DBeaver [12] által generált DDL utasítások felhasználásával.

3.3.4. Az adatbázis táblái

Az ER diagramon ábrázolt adatbázistáblák a relációs adatábrázolás és a felelősségi körök szétválasztásának elveit követve kerültek kialakításra. A rendszer entitásai a következők:

- **auth:** A hitelesítési adatok és a felhasználói fiókok biztonságos tárolásáért felelős tábla. Tartalmazza a rekord egyedi azonosítóját (id), az egyedi (UNIQUE kényszerrel védett) felhasználónevet (username) és a bejelentkezéshez szükséges jelszót (password). Kizárólag az autentikációs folyamatok során van szerepe.
- **player:** A játékosok publikus profiladatait tartalmazó tábla. Minden bejegyzése pontosan egy hitelesítési fiókhoz kapcsolódik az `auth_id` idegen kulcson keresztül (1:1 kapcsolat). A profil létrehozásának idején (`created_at`) felül redundánsan tartalmazza a felhasználónevet is.

- **stats:** Az egyes játékosokhoz tartozó globális és összesített statisztikákat nyilvántartó entitás. A `player_id` idegen kulcs segítségével rendelődik hozzá a megfelelő profilhoz. Célja a teljesítménykövetés: tárolja az eddig lejátszott mérkőzések számát (`total_games`), a megszerzett győzelmeket (`total_wins`) és az összesített meccspontokat (`total_points`).
- **history:** A korábban lejátszott mérkőzések történetét és eredményeit rögzítő tábla, amely egy-a-többhöz (1:N) kapcsolatban áll a **player** táblával. Minden lezárt mérkőzés után új rekord jön létre benne, amely megőrzi a mérkőzés időpontját (`match_date`), az aktuális ellenfél nevét (`opponent_name`), a meccs végkimenetelét (`match_status`), valamint az elért végeredményt (`final_score`).

3.4. Backend architektúra és infrastruktúra

3.4.1. Fontosabb metódusok dokumentációja

A játékmotor belső működésének megértéséhez elengedhetetlen a kulcsfontosságú metódusok pontos definíciója. Az alábbiakban a rendszer legfontosabb ellenőrző logikáját mutatom be.

Az Engine osztály <code>isValidMove</code> metódusának megvalósítása:	
Bemenő adat	<code>player</code> : Player – a lépést végrehajtó játékos objektuma; <code>cardToPlay</code> : Card – a kijátszani kívánt kártyalap; <code>ledCard</code> : Card – az ellenfél által hívott (már az asztalon lévő) kártyalap
Kimenő adat	boolean – <code>true</code> , ha a kártya kijátszása a szabályoknak megfelel; <code>false</code> , ha az adott szituációban a lépés érvénytelen
Tevékenység	A metódus a snapszer szigorú lépésszabályait érvényesíti. Ha a pakli nem üres és nincs takarva, minden lépést engedélyez. Zárt vagy elfogyott pakli (végjáték) esetén sorrendben ellenőrzi a kötelező érvényű szabályokat: 1. Színkényszer (ha van a hívott színből a kézben, azonos színt kell tenni), 2. Felülütési kényszer (ha van erősebb lap az adott színből, azt kötelező megjátszani), 3. Adukényszer (ha nincs a hívott színből, de van adu a kézben, akkor kötelező adut tenni). Ha a játékos sem azonos színű lappal, sem aduval nem rendelkezik, bármilyen lapot érvényesen eldobhat.

3.2. táblázat. Az `isValidMove` metódus működésének részletezése a tényleges algoritmus alapján

Az Engine osztály evaluateStrongerCard metódusának megvalósítása:	
Bemenő adat	ledCard: Card – az ütésben elsőként kijátszott (hívott) kártyalap; followedCard: Card – a másodikként megjátszott kártyalap
Kimenő adat	Card – a két lap közül a játékszabályok értelmében erősebb, azaz az ütést elvivő kártyalap objektuma
Tevékenység	Meghatározza az ütés nyertesét a két kijátszott lap összehasonlításával. Ha a két kártya azonos színű, a magasabb belső értékű (rangú) lapot adja vissza. Ha a színek eltérnek, de a másodikként megjátszott lap megegyezik az aktuális aduszínnel (trumpSuit), akkor ez az adalap nyeri az ütést. Minden egyéb esetben (vagyis ha a második lap eltérő színű, de nem adu), az elsőként kijátszott lap marad az erősebb.

3.3. táblázat. Az evaluateStrongerCard metódus működésének részletezése a tényleges algoritmus alapján

Az Engine osztály switchTrumpCard metódusának megvalósítása:	
Bemenő adat	player: Player – az aducserét kezdeményező játékos objektuma
Kimenő adat	Object – egy objektum, amely tartalmaz egy success (boolean) tulajdonságot a művelet sikerességéről, illetve sikertelen kísérlet esetén egy magyarázó message (string) hibaüzenetet is tartalmaz
Tevékenység	A metódus az aducsere logikáját oldja meg. Először ellenőrzi, hogy a pakli nincs-e lezárva (takarva), mivel a takarás után már nem lehet adut cserélni. Utána ellenőrzi, hogy az adott játékos nem null és ő van-e soron. Ezután tovább ellenőrzi, hogy van-e az adu színnel ellátott alsó a kezében, ha ezek a feltételek igazak akkor visszatér egy objektummal, amiben egy igaz boolean szerepel, ha nem akkor hamis boolean-al ellátott objektummal tér vissza a metódus.

3.4. táblázat. A switchTrumpCard metódus működésének részletezése a tényleges algoritmus alapján

A Room osztály broadcastState metódusának megvalósítása:	
Bemenő adat	Nem rendelkezik bemeneti paraméterrel, azonban a Room és az Engine osztályok belső állapotát használja fel az adatok kinyeréséhez.
Kimenő adat	Klasszikus visszatérési értéke nincs (void), viszont a hálózati kapcsolaton keresztül aszinkron módon egy gameStateUpdate eseményt bocsát ki a kliensek felé.
Tevékenység	Ez a metódus felel azért, hogy mindkét kliensnek szinkronizált játékelülete legyen egyidejűleg. A rendszer végigiterál az Engine osztályban tárolt két játékoson, hogy mindenkinek egy perszonalizált adatcsomagot (gameState) generáljon. Biztonságos marad a hálózati kommunikáció, hiszen az összeállított objektumba nem kerülnek be az ellenfél rejtett kártyái, hanem az csak a játékos saját lapjait, a publikus pontszámokat, illetve a játéktér látható elemeit (adulap, pakli mérete) tartalmazza. Minden egyes eseményt követően, ami a játékelületet befolyásolja, ez a metódus lefut és frissíti a kliensek állapotát. A végrehajtás utolsó lépéseként a generált csomag kiküldésre kerül az adott felhasználó egyedi socketId-jére.

3.5. táblázat. A broadcastState metódus működésének részletezése a tényleges algoritmus alapján

3.4.2. A szerveroldali belépési pont (server.js)

A `server.js` fájl az alkalmazás backend központi belépési pontja. Ez az állomány felelős a webszerver elindításáért, a kliens és a szerver közötti hálózati kapcsolatok felépítéséért, valamint a játék logikáját és az adatbázist összekötő hírendszert működtetéséért. A modul a Node.js környezetben fut, és több kulcsfontosságú technológiát (Express.js [13], Socket.IO [6], Sequelize [5]) integrál egyetlen egységes rendszerbe.

A modul legfontosabb felelősségi körei a következők:

- **Szerver inicializálása és REST API végpontok:** Az Express.js keretrendszer segítségével itt történik a HTTP szerver konfigurálása, a CORS (Cross-Origin Resource Sharing) házirend beállítása, valamint az alapvető REST API végpontok (regisztráció, bejelentkezés, statisztikák lekérése) útvonalválasztóinak (routing) bekötése. Továbbá ez a rész felelős a lefordított kliensoldali fájlok (statikus frontend) kiszolgálásáért is.

- **Hitelesítés és jogosultságkezelés:** A játékosok csatlakozásakor a szerver JSON Web Token (JWT) segítségével azonosítja a regisztrált felhasználókat. Vendég (nem bejelentkezett) játékosok esetén ellenőrzi, hogy a választott név nem ütközik-e már létező fiókkal, illetve nem játszik-e éppen valaki ugyanazzal a névvel a szerveren, ezzel elkerülve a névütközéseket.
- **Várólista (Matchmaking) és szobakezelés:** A szerver egy memóriában tárolt várólistán (queue) tartja nyilván a meccskereső játékosokat. Amint legalább két játékos összegyűlik, a rendszer automatikusan párosítja őket, létrehoz egy egyedi azonosítóval ellátott szobát (egy Room példányt), és mindkét klienst ebbe a zárt Socket.IO csatornába irányítja át.
- **Valós idejű eseménykezelés (WebSockets):** A folyamatban lévő mérkőzések során a szerver folyamatosan hallgatja a kliensektől érkező eseményeket (például kártyarakás: `playerMove`, aducsere: `switchTrumpCard`, takarás: `close`), és azokat feldolgozásra a megfelelő szobapéldány játékmotorjának (Engine) továbbítja.
- **Kapcsolatszakadás és adatperzisztencia:** A rendszer robusztusan kezeli a váratlan hálózati megszakadásokat (`disconnect` esemény). Ha egy játékos kilépés vagy internetes hiba miatt elhagyja a folyamatban lévő meccset, a szerver a bent maradt játékost hirdeti ki győztesnek. A mérkőzések végeztével a `server.js` a Sequelize ORM segítségével aszinkron módon elmenti a meccs eredményét a `History` táblába, frissíti a résztvevők összesített statisztikáit a `Stats` táblában, végül pedig felszabadítja a lezárt szoba által lefoglalt memóriát.

3.4.3. REST API végpontok dokumentációja

Kommunikációs hidat képeznek a REST API végpontok a frontend és a backend adatbázis között. A beérkező kérések feldolgozását és az adatáramlás irányítását a vezérlő (controller) funkciók végzik. JSON Web Token (JWT) [14] alapú hitelesítés védi a privát végpontokat, így a szenzitív adatokhoz (mint a profil statisztikái) csak és kizárólag a megfelelő jogosultsággal rendelkező felhasználók férhetnek hozzá.

Metódus	Végpont (Endpoint)	Funkció és leírás
POST	/api/auth/register	Új felhasználói fiókot hoz létre az adatbázisban. Ellenőrzi a jelszó és a név minimális és maximális hosszát, majd sikeres validáció után egy adatbázis tranzakció keretében szinkronban rögzíti az Auth és Player modelleket.
POST	/api/auth/login	Hitelesíti a kliens által beküldött belépési adatokat. A bcrypt algoritmussal összehasonlítja a titkosított jelszavakat, majd sikeres belépés esetén egy 30 napig érvényes, aláírt JWT tokenel tér vissza.
GET	/api/stats/getStats	Lekérdezi egy adott játékos összesített profiladatait. Kizárólag érvényes JWT tokenel hívható, amelyből a szerver visszafejti a felhasználó azonosítóját, majd válaszként szolgáltatja a Stats és a History táblákból a legutóbbi tíz mérkőzés eredményét.

3.6. táblázat. A fontosabb REST API végpontok és vezérlők specifikációja

3.4.4. Rendszeresemények naplózása (Logging)

A szerveroldali működés nyomon követése, az anomáliák kiszűrése és a hibák hatékony diagnosztizálása érdekében az alkalmazás egy egyedi, fájlalapú naplózó (logger) modult alkalmaz. A weboldal ezen komponense a Node.js beépített fájlkezelő (**fs**) és útvonalválasztó (**path**) moduljaira épül, így nem igényel harmadik féltől származó, külső függőséget.

A naplózó modul főbb jellemzői és szoftvertervezési megoldásai a következők:

- **Automatikus könyvtárkezelés:** A modul inicializálásakor ellenőrzi a naplófájlok számára dedikált "logs" könyvtár meglétét az operációs rendszerben. Amennyiben a mappa nem létezik, a script (**fs.mkdirSync**) automatikusan létrehozza azt, megelőzve az Input/Output hibákat.
- **Strukturált adatrögzítés:** Az exportált **logMatchSearch** függvény egy egysegű, könnyen olvasható formátumban rögzíti az eseményeket. Minden bejegyzés tartalmazza a művelet pontos, lokalizált időbélyegét, a játékos nevét, valamint a csatlakozási kísérlet státuszát.

- **Hibakövetés:** A modul megkülönbözteti a sikeres és sikertelen meccskeresési eseményeket. Hibás kísérletek esetén (például érvénytelen JWT token, foglalt név, vagy hibás adatbázis kapcsolat) a naplósor kiegészül a hiba pontos okával is, ami elengedhetetlen a későbbi rendszerüzemeltetési feladatokhoz.

3.4.5. Alkalmazott szoftverkönyvtárak és függőségek

A projekt fejlesztése során a `package.json` fájlban meghatározott, célirányosan kiválasztott külső könyvtárak (függőségek) kerültek felhasználásra. A beépülő alkönyvtárak (transzitiv függőségek) mellőzésével a rendszer legfőbb építőelemei és azok felelősségi körei a következők:

- **Express (express v5.2.1):** A Node.js legelterjedtebb minimalista webes keretrendszere, amely a REST API végpontok kialakításáért, a HTTP kérések (útvonalválasztás és middleware-ek) kezeléséért, valamint a statikus frontend fájlok kiszolgálásáért felel [13].
- **Socket.IO (socket.io v4.8.3):** A valós idejű, kétirányú hálózati kommunikációt (WebSockets alapokon) biztosító keretrendszer, amely a folyamatos és gyors adatcserét, ezáltal a tényleges játékmenet szinkronizálását végzi a kliensek és a szerver között [6].
- **Sequelize (sequelize v6.37.8):** Egy fejlett, ígéret-alapú (promise-based) ORM (Object-Relational Mapper) eszköz, amely az adatbázis-műveleteket absztrakt, objektumorientált módon kezeli, kiváltva a nyers SQL lekérdezések manuális írását [5].
- **PostgreSQL csomagok (pg v8.20.0, pg-hstore v2.3.4):** A Sequelize ORM számára elengedhetetlen natív adatbázis-meghajtók, amelyek a Node.js és a PostgreSQL adatbázis-szerver közötti alacsony szintű kommunikációt valósítják meg [15].
- **JSON Web Token (jsonwebtoken v9.0.3):** A token alapú, állapotmentes (stateless) felhasználói hitelesítést (autentikációt) és jogosultságkezelést biztosító könyvtár, amely titkosított formában tárolja a munkamenet adatait [14].

- **Bcrypt.js (bcryptjs v3.0.3):** A felhasználói jelszavak biztonságos, egyirányú titkosításáért (hashing) felelős modul, amellyel garantálható, hogy az adatbázis kompromittálódása esetén sem nyerhetők ki a jelszavak nyílt szöveggént [16].
- **CORS (cors v2.8.6):** Biztonsági köztesréteg (middleware) az Express-hez, amely a Cross-Origin Resource Sharing házirend konfigurálásával engedélyezi az eltérő tartományokból érkező böngészős kéréseket [17].
- **Dotenv (dotenv v17.2.3):** A környezeti változók (például adatbázis-kapcsolati azonosítók, titkosító kulcsok, szerver portok) egy elkülönített `.env` fájlból történő, biztonságos betöltéséért felel [18].
- **Tailwind CSS (tailwindcss v4.1.18, @tailwindcss/vite):** A kliensoldali grafikus felület (UI) atomi osztályokon alapuló, reszponzív formázását végző CSS keretrendszer és annak build-eszköz (Vite) integrációja [7].
- **Nodemon (nodemon v3.1.14):** Fejlesztési célú segédeszköz (devDependency), amely a forráskód mentésekor automatikusan újraindítja a Node.js háttérszerver, ezzel jelentősen gyorsítva az iterációs folyamatot [19].

3.5. Fejlesztői környezet kialakítása és futtatás helyi környezetben

A projekt lokális környezetben történő futtatása és a továbbfejlesztés megkezdése előtt elengedhetetlen a megfelelő szoftveres infrastruktúra kialakítása. Ez a fejezet lépésről lépésre bemutatja a szükséges előfeltételeket, valamint a szerveroldali (backend) és a kliensoldali (frontend) komponensek elindításának folyamatát.

3.5.1. Előfeltételek

Az alkalmazás futtatásához a fejlesztői gépen az alábbi szoftverek és eszközök megléte elengedhetetlen:

- **Node.js:** A JavaScript futtatókörnyezet, amely magában foglalja az NPM (Node Package Manager) csomagkezelőt is. Az aszinkron funkciók és a felhasznált könyvtárak miatt legalább a 18-as vagy újabb verzió ajánlott [20].

- **PostgreSQL:** Egy lokálisan futó, vagy egy virtuális (pl. Docker konténeres) környezetben elérhető relációs adatbázis-szerver, amely a felhasználói adatok és a statisztikák tárolásáért felel.

3.5.2. A Backend környezet beállítása

A szerveroldali alkalmazás beüzemelése a projekt `backend` mappájában történik. A folyamat az alábbi lépésekből áll:

1. **Függőségek telepítése:** A parancssorban kiadott `npm install` parancs letölti és telepíti a `package.json` fájlban definiált összes szükséges szoftverkönyvtárat (pl. Express, Socket.IO, Sequelize).
2. **Környezeti változók konfigurálása:** A gyökérkönyvtárban létre kell hozni egy `.env` fájlt, amely tartalmazza az alábbiakat:
 - `db_username=snapszeradmin`: Az adatbázis adminisztrátori azonosítója.
 - `db_pass=<ADATBAZIS_JELSZO>`: A hozzáféréshez szükséges biztonsági jelszó helyőrzője.
 - `PORT=3000`: Az alkalmazásszerver futtatási portja.
 - `DATABASE_URL=postgres://snapszeradmin:<ADATBAZIS_JELSZO>@snapszer-db.postgres.database.azure.com:5432/postgres?sslmode=verify-full`: A teljes PostgreSQL kapcsolati karakterlánc az SSL titkosítás kényszerítésével.
 - `JWT_SECRET=<EGYEDI_TITKOS_KULCS>`: A munkamenetek hitelesítésére szolgáló egyedi kriptográfiai kulcs (pl. `b88d4292...` kezdetű hash).

Értelemszerűen a fenti listában szereplő paraméterek a saját Azure-adatbázisom eléréséhez lettek konfigurálva. Az alkalmazás más környezetben történő telepítésekor ezeket az értékeket a saját adatbázis-szolgáltató által biztosított adatokkal kell helyettesíteni.

3. **Szerver indítása:** Az `npm run dev` parancs futtatásával a Nodemon segédprogram elindítja a háttérszolgáltatást. Sikeres indulás esetén a konzolon megjelenik a figyelési port száma, valamint az adatbázishoz való sikeres csatlakozás ténye.

3.5.3. A Frontend környezet beállítása

A kliensoldali felület a modern, és optimalizált Vite build-eszközre épül [21]. Ennek indítása a **frontend** könyvtárból történik a következő módon:

1. **Függőségek telepítése:** Hasonlóan a backendhez, az `npm install` parancs futtatásával telepíthetők a React, a Tailwind CSS és egyéb kliensoldali függőségek.
2. **Fejlesztői szerver indítása:** Az `npm run dev` parancs kiadásával a Vite elindítja a lokális fejlesztői szerveret. A rendszer azonnali frissítést (Hot Module Replacement) biztosít, így a kódmódosítások azonnal, oldalfrissítés nélkül megjelennek a böngészőben.

Külön kiemelendő a frontend hálózati kommunikációjának környezetfüggetlen kialakítása. A valós idejű kapcsolatot biztosító Socket.IO kliens inicializálása során a rendszer nem fixen beégetett IP címet használ a szerver azonosítására, hanem a böngésző beépített `window.location.origin` tulajdonságára támaszkodik.

3.6. Rendszertesztelés és a megvalósult használati esetek

A fejlesztési fázis lezárultával a szoftver minőségbiztosítása és az üzleti logika helyességének ellenőrzése következett. Ennek alapját a Behaviour-Driven Development (BDD) módszertan adta, amely a rendszer működését a felhasználói viselkedés felől közelíti meg.

Az alábbi táblázat az alkalmazás legfontosabb használati eseteit (Use Case) és azok tesztelésének eredményeit mutatja be a szabványos *Given-When-Then* (Adott-Amikor-Akkor) formátumban. A jobb oldali oszlopban található „OK” minősítés igazolja, hogy a végleges implementáció a tesztek során az elvárt viselkedést produkálta, így az aktuális megvalósítás fedi az előzetes terveket.

3.7. táblázat. A rendszer használati eseteinek részletezése és teszt eredményei

#	Eset	Leírás		Eredmény
1.	Sikeres regisztráció	GIVEN	Vendégként tartózkodom a regisztrációs oldalon.	OK
		WHEN	Kitöltöm a szükséges adatokat (felhasználónév, jelszó) és rákattintok a „Regisztráció” gombra.	
		THEN	A rendszer létrehozza a fiókomat az adatbázisban.	
2.	Sikeres bejelentkezés	GIVEN	Vendégként tartózkodom a bejelentkezési oldalon.	OK
		WHEN	Megadom a regisztrált felhasználónevemet és jelszavamat, majd a „Belépés” gombra kattintok.	
		THEN	A rendszer hitelesíti az adatokat, generálja a token-t, és a főmenübe irányít.	
3.	Játék keresése (Matchmaking)	GIVEN	Játékosként be vagyok jelentkezve, és a főmenüben vagyok.	OK
		WHEN	Rákattintok a játék keresése gombra.	
		THEN	A rendszer felvesz a várólistára, majd amint van elérhető ellenfél, létrehoz egy közös szobát.	
4.	Játék keresése egy foglalt névvel	GIVEN	Vendégként vagyok a főmenüben.	OK
		WHEN	Beírom a becenevemet a keresőbe és rákattintok a játék keresése gombra.	
		THEN	A rendszer észreveszi, hogy már játszik egy ilyen nevű játékos és jelzni nekem a főmenüben, hogy foglalt a név.	

#	Eset	Leírás		Eredmény
5.	Érvényes kártya kijátszása	GIVEN	Játékosként egy mérkőzésben vagyok, és én következem.	OK
		WHEN	A kezemben lévő kártyák közül az ütéskenyszer vagy színenyszer szabályainak megfelelően kijátszok egyet.	
		THEN	A rendszer elfogadja a lépést, a lap megjelenik az asztalon, és a sor az ellenfélre száll.	
6.	Érvénytelen kártya kijátszása	GIVEN	Játékosként egy mérkőzésben vagyok, és én következem.	OK
		WHEN	A kezemben lévő kártyák közül az ütéskenyszer vagy színenyszer szabályainak nem megfelelően kijátszok egyet.	
		THEN	A rendszer elutasítja a lépést, a lap egy vastag piros keretet kap és rezeg egy rövid ideig jelezve, hogy a lépésem illegális.	
7.	Bemondás (20/40)	GIVEN	Hívó félként van egy királyom és egy felsőm azonos színből.	OK
		WHEN	Kijátszom a 20-at vagy 40-et érő páros egyik tagját (királyt vagy felsőt) a gombra kattintva.	
		THEN	A rendszer érvényesíti a bemondást, jóváírja a plusz pontokat, és megjeleníti a lapot.	
8.	Ütés kiértékelése	GIVEN	Mindkét játékos kijátszotta a kártyáját az adott körben.	OK
		WHEN	A rendszer megvizsgálja a lapokat az aduszín és a laperősség alapján.	

#	Eset	Leírás		Eredmény
		THEN	A rendszer megállapítja az ütés nyertesét, jóváírja a pontokat, és frissíti a klienseket.	
9.	Aducsere	GIVEN	Játékosként én következem, és a kezemben van az aktuális aducszínhez tartozó alsó kártyalap.	OK
		WHEN	Rákattintok a játéktéren elhelyezett aducslapra.	
		THEN	A rendszer kicseréli az aducslapot a kezemben lévő alsóra, és frissíti a játéktér állapotát mindkét kliensnél.	
10.	Helytelen aducsere	GIVEN	Játékosként én következem, és a kezemben nincs az aktuális aducszínhez tartozó alsó kártyalap.	OK
		WHEN	Rákattintok a játéktéren elhelyezett aducslapra.	
		THEN	A rendszer elutasítja az eseményt és az aducslap egy vastag piros keretet kap és rezeg egy rövid ideig.	
11.	Pakli takarása (Zárás)	GIVEN	Játékosként én vagyok soron, és a húzópakliban (talonban) még legalább 3 kártyalap található.	OK
		WHEN	Egy gombnyomással kezdeményezem a pakli takarását.	
		THEN	A rendszer lezárja a paklit, vizuálisan jelzi ezt a talonon, és egy felugró ablakban értesíti a résztvevőket.	
12.	Kör megnyerése (66 pont)	GIVEN	Egy mérkőzés során véget ér egy ütés.	

#	Eset	Leírás		Eredmény
		WHEN	Az ütet elvivő játékos összegyűjtött pontjainak száma eléri a 66-ot.	OK
		THEN	A rendszer lezárja a kört, kihirdeti a nyertest, és kiosztja a megfelelő meccspontokat.	
13.	Kör elvesztése (66 pont az ellenfélnek)	GIVEN	Egy mérkőzés során véget ér egy ütés.	OK
		WHEN	Az ellenfelem az általa elvitt ütésekkel összegyűjt legalább 66 pontot.	
		THEN	A rendszer lezárja a kört, az ellenfelet hirdeti ki a kör nyertesének, és neki oszt meccspontot.	
14.	Meccs megnyerése (7 meccspont)	GIVEN	Egy aktív mérkőzés során a rendszer kiértékel egy újonnan megnyert kört.	OK
		WHEN	A frissen megnyert körért kapott pontokkal az én összesített meccspontjaim száma eléri a 7-et.	
		THEN	A rendszer lezárja a mérkőzést, kihirdet engem győztesnek, és elemi az eredményt az adatbázisba.	
15.	Meccs elvesztése (7 meccspont az ellenfélnek)	GIVEN	Egy aktív mérkőzés során a rendszer kiértékel egy elvesztett kört.	OK
		WHEN	Az ellenfelem megnyeri a kört, és az azért kapott pontokkal az ő meccspontjainak száma eléri a 7-et.	

#	Eset	Leírás		Eredmény
		THEN	A rendszer a mérkőzést lezárja, az ellenfelet hirdeti ki győztesnek, engem pedig a főmenübe irányít.	
16.	Játék elhagyása (Disconnect)	GIVEN	Egy aktív mérkőzésen veszek részt.	OK
		WHEN	Kilépek az oldalról vagy bezárom a böngészőt.	
		THEN	A rendszer érzékeli a kilépést, a bent maradt játékost hirdeti ki győztesnek, és lezárja a szobát.	
17.	Statisztikák megtekintése	GIVEN	Játékosként bejelentkezve a főmenüben tartózkodom.	OK
		WHEN	Rákattintok a statisztikák gombjára.	
		THEN	A rendszer lekéri és listázza az eddig elért eredményeimet (győzelmek, meccsek száma).	
18.	Játék keresés megszakítása	GIVEN	A főmenüben tartózkodom és a játék keresése zajlik.	OK
		WHEN	Rákattintok az "X" gombra a játék keresésnél.	
		THEN	A játék keresése megszakad és újra megjelenik a játék keresése gomb.	

3.6.1. Egység tesztek

A rendszer üzleti logikájának hibamentes működését egységes tesztek biztosítják. A szerveroldali teszteléshez a népszerű Jest [22] keretrendszert használtam, amely beépített asszertációkkal és mockolási (függőség szimulálás) lehetőségekkel támogatja a hatékony tesztelést.

Az architektúra objektumorientált felépítéséből adódóan minden központi osztály saját, dedikált tesztfájlt kapott. Ezek az esetek teljesen elszigetelten vizsgálják

a metódusok működését, az extrém szituációs bemeneteket (edge cases), valamint a játékszabályok (például szín és ütékenységyszer) maradéktalan érvényesülését.

A 3.15. ábra a tesztkészlet futtatásának végeredményét szemlélteti a fejlesztői környezet termináljában, megerősítve, hogy az összes teszteset hibátlanul (zöld státusszal) lefutott.

```
Test Suites: 5 passed, 5 total
Tests:      44 passed, 44 total
Snapshots:  0 total
Time:       0.437 s, estimated 1 s
Ran all test suites.
```

3.15. ábra. Az egységtesztek sikeres lefutása a parancssorban

Az ábra igazolja, hogy a Jest mindössze 0,437 másodperc alatt hibátlanul lefutatta mind az 5 tesztkészletet és az abban foglalt összesen 44 egységtesztet.

Az alábbi táblázatok osztályonként (tesztfájlonként) részletezik a vizsgált eseteket és azok lefutási eredményeit.

3.8. táblázat. A Card.test.js tesztesetei

#	Teszteset leírása	Státusz
1.	A <code>constructor</code> segítségével történő példányosítás helyességének ellenőrzése a megadott szín, típus és pontértékek alapján.	OK
2.	Az <code>isTrump()</code> metódus által végzett adalap-felismerés validálása a megadott aktuális aduszín alapján.	OK

3.9. táblázat. A Deck.test.js tesztesetei

#	Teszteset leírása	Státusz
1.	Annak ellenőrzése, hogy a <code>constructor</code> hívása után az <code>isEmpty()</code> metódus valóban üres paklit jelez-e.	OK
2.	A <code>generate()</code> metódus által létrehozott kártyák pontos számának (20 darab) és típusának ellenőrzése.	OK
3.	A <code>drawCard()</code> hívásakor bekövetkező kártyahúzás és a pakli méretcsökkenésének validálása.	OK

#	Teszteset leírása	Státusz
4.	Az <code>isEmpty()</code> metódus helyes logikai visszatérési értékének vizsgálata a pakli teljes kiürülése után.	OK
5.	A <code>shuffle()</code> metódus futtatása után a pakli méretének (20 lap) változatlanságának ellenőrzése.	OK
6.	A <code>shuffle()</code> algoritmus hatékonyságának vizsgálata a legfelső lapok vizsgálata alapján.	OK

3.10. táblázat. A `Player.test.js` tesztesetei

#	Teszteset leírása	Státusz
1.	A <code>constructor</code> által beállított alapértékek (név, <code>socketId</code> , pont) helyességének ellenőrzése.	OK
2.	A <code>drawCard()</code> metódus működésének validálása a kártya játékos kezébe kerülése alapján.	OK
3.	Az <code>addRoundPoints()</code> és <code>resetRoundPoints()</code> metódusok által végzett pontkezelés és nullázás ellenőrzése.	OK
4.	A <code>clearHand()</code> metódus által végzett teljes kézürítés helyességének vizsgálata.	OK
5.	A <code>clearWonCards()</code> hívása után a megnyert lapok listájának törlődésének ellenőrzése.	OK
6.	Az <code>addCardsToWonCards()</code> metódus tesztelése pontosan két kártya sikeres hozzáadásával.	OK
7.	Az <code>addCardsToWonCards()</code> metódus hibaágának ellenőrzése érvénytelen (üres) lista megadásakor.	OK
8.	A <code>removeCard()</code> metódus által végzett kártyaeltávolítás validálása index alapján.	OK
9.	A <code>getAnnouncements()</code> metódus tesztelése érvényes 20 pontos bemondás felismerésével normál színből.	OK
10.	A <code>getAnnouncements()</code> metódus tesztelése érvényes 40 pontos bemondás felismerésével aduszínből.	OK
11.	A <code>getAnnouncements()</code> validálása üres visszatérési értékre, ha nincs bemondható pár a kézben.	OK

#	Teszteteset leírása	Státusz
12.	A <code>playAnnouncement()</code> metódus sikerességének ellenőrzése az érték és a kijátszott kártya visszaadása alapján.	OK
13.	A <code>playAnnouncement()</code> metódus hibaágának vizsgálata érvénytelen bemonadás kísérletekor.	OK
14.	A <code>hasSuit()</code> metódus által végzett színellenőrzés helyességének validálása.	OK
15.	A <code>hasTrump()</code> metódus tesztelése az aduszínű kártya meglétének felismerésére.	OK
16.	A <code>hasStrongerCard()</code> metódus validálása az erősebb lap meglétének felismerésére az ütéskényszerhez.	OK

3.11. táblázat. Az `Engine.test.js` teszttesetei

#	Teszteteset leírása	Státusz
1.	Az <code>Engine constructor</code> általi helyes inicializálásának ellenőrzése a játékosokkal és a socket mockkal.	OK
2.	A <code>startRound()</code> metódus által végzett lapkiosztás és adubeállítás validálása.	OK
3.	Az aktív játékos cseréjének ellenőrzése a <code>startRound()</code> metódus ismételt hívásakor.	OK
4.	A <code>switchTrumpCard()</code> metódus sikerességének vizsgálata érvényes aducsere esetén.	OK
5.	A <code>switchTrumpCard()</code> metódus elutasításának ellenőrzése, ha a pakli már le van zárva.	OK
6.	Az <code>evaluateStrongerCard()</code> metódus által végzett kártyakiértékelés validálása az adu és a laperősség alapján.	OK
7.	Az <code>isValidMove()</code> metódus tesztelése bármely lap kijátszásának engedélyezésére nyitott paklinál.	OK
8.	Az <code>isValidMove()</code> által végzett szín- és ütéskényszer ellenőrzés validálása zárt pakli esetén.	OK
9.	Az <code>isValidMove()</code> által kényszerített aduhasználat ellenőrzése zárt paklinál.	OK

#	Teszteteset leírása	Státusz
10.	A <code>close()</code> metódus által rögzített zárási adatok helyességének vizsgálata érvényes takaráskor.	OK
11.	A <code>calculateMatchPoints()</code> metódus tesztelése a megszokott takarás nélküli győzelmi pontok kiszámítására.	OK
12.	A <code>calculateMatchPoints()</code> validálása a takaró játékos győzelme esetén.	OK
13.	A <code>calculateMatchPoints()</code> tesztelése a takaró játékos büntetésének kiszámítására veszteség esetén.	OK
14.	Az <code>endRound()</code> metódus működésének és a teljes mérkőzés lezárásának (7 pont) ellenőrzése.	OK

3.12. táblázat. A Room.test.js tesztesetei

#	Teszteteset leírása	Státusz
1.	A <code>Room constructor</code> általi helyes inicializálásának ellenőrzése a kliens és játékos adatokkal.	OK
2.	A <code>startGame()</code> metódus által kiváltott motorindítás és hálózati állapotközvetítés validálása.	OK
3.	A <code>handleSwitchTrumpCard()</code> metódus sikeres lefutásának és a frissített állapot kiküldetésének ellenőrzése.	OK
4.	Az érvénytelen aducserre esetén a <code>handleSwitchTrumpCard()</code> által küldött hibaüzenet validálása.	OK
5.	A <code>handleMove()</code> metódus által kezelt mérkőzésvég ellenőrzése azonnali bemondással történő győzelem esetén.	OK
6.	A <code>broadcastState()</code> metódus által összeállított és kiküldött teljes játékállapot pontosságának vizsgálata.	OK

3.6.2. Tesztelés által befolyásolt döntések

A szoftver fejlesztése során a folyamatos manuális tesztelés több olyan hiányosságra is rávilágított, amelyek nem feltétlenül programhibák voltak, hanem főleg a felhasználói élményt és a játékmenet érthetőségét rontották. Ezen visszajelzések alapján több kritikus módosítást is eszközöltem a rendszeren:

- **Vizuális visszacsatolás a játékmenet eseményeiről:** A kezdeti tesztfázisban a pakli takarása (zárása) vagy a 20/40 pontos bemondások során az ellenfél nem kapott egyértelmű jelzést arról, hogy mi történt, így a játékmenet zavarossá vált. Ezt orvosolva bevezettem egy értesítési rendszert, amely azonnal tájékoztatja a másik felet ezekről a stratégiai lépésekről, biztosítva a játék folyamatos követhetőségét. Ezeket az értesítéseket úgynevezett felugró ablakokkal oldottam meg, amelyek színnel is adnak információt: pl. narancssárga jelenik meg, amikor az ellenfél játszik ki 20/40-et és kékkel ha mi játszunk ki.
- **A körök kiértékelésének ütemezése:** Eredetileg, amikor a második játékos is kijátszotta a kártyáját, a rendszer azonnal elvégezte a kiértékelést, és a lapok azonnal eltűntek az asztalról. Ez zavaró volt, mivel a felhasználóknak nem maradt idejük vizuálisan feldolgozni az ütést. Ennek javítására bevezettem egy 2 másodperces késleltetést, amely alatt mindkét kijátszott kártya látható marad. Ez a rövid szünet segít a játékosoknak követni a pontszámítást és az ütés irányát, mielőtt a rendszer továbblépne a következő körre.
- **A várólistáról való kilépés lehetősége:** A hálózati kommunikáció tesztelésekor kiderült, hogy miután a játékos elindította a keresést, nem volt lehetősége meggondolnia magát és elhagyni a várólistát. Ez kényelmetlen helyzetet teremtett hosszú várakozási idő esetén. A probléma megoldására a kereső felületet kiegészítettem egy "X" (megszakítás) gombbal, amely lehetővé teszi a keresési folyamat biztonságos leállítását és a főmenübe való visszatérést anélkül, hogy az oldalt újra kellene tölteni.

Ezek a módosítások jelentősen növelték a szoftver ergonómiáját és a játékelmény folytonosságát, bizonyítva a tesztelés fontosságát.

4. fejezet

Összegzés

A szakdolgozat keretein belül sikeresen megtervezésre és implementálásra került egy valós idejű, böngészőből játszható kétszemélyes snapszer alkalmazás. A modern webes technológiák (React/Vite, Node.js, Socket.IO, PostgreSQL) és a felhőalapú architektúra alkalmazásával a szoftver stabil, reszponzív és azonnal használható élményt nyújt a felhasználók számára. Bár a rendszer jelenlegi állapotában is egy teljes értékű, élesített szoftver, a moduláris kialakítás számos lehetőséget biztosít a jövőbeli bővítésekre.

4.1. Továbbfejlesztési lehetőségek

A kliensoldali felület és a szerveroldali üzleti logika (Engine) szigorú szétválasztása kiváló alapot ad ahhoz, hogy az alkalmazás funkcióit a későbbiekben új játékmódokkal gazdagítsuk. A legfőbb továbbfejlesztési irányok a következők:

4.1.1. Többjátékos (3 és 4 fős) játékmódok integrálása

Bár a snapszer alapvetően kétszemélyes játékként a legismertebb, léteznek rendkívül népszerű három és négyszemélyes változatai is. Ezek megvalósítása a rendszer egy logikus lépése lenne:

- **Üzleti logika bővítése:** A szerveroldali `Engine` osztályt fel kell készíteni két-tőnél több `Player` példány egyidejű kezelésére. Át kell alakítani a köralapú léptetést, valamint integrálni kell a többjátékos verziók egyedi szabályait (például a teljes pakli kiosztását talon nélkül, a hívásokat, vagy a csapattársak pontjainak közös számítását 4 fős mód esetén).

- **Felhasználói felület adaptációja:** A frontend komponenseket úgy kell áttervezni, hogy az asztalon dinamikusan, vizuálisan is átláthatóan helyezze el az új ellenfeleket (például a képernyő jobb és bal oldalán, illetve a tetején).

4.1.2. Egyjátékos mód: Játék "Bot" ellen

Annak érdekében, hogy a felhasználók várakozás nélkül, gyakorlás céljából is játsz hassanak abban az esetben is, ha épp nincs elérhető emberi ellenfél a várólistán, kiemelten hasznos lenne egy kód által vezérelt gépi ellenfél (Bot) bevezetése:

- **Döntéshozatali algoritmusok:** Első lépésben egy egyszerűbb, szabályalapú gép is elegendő lehet, amely pusztán az érvényes lépések közül választ, előtérbe helyezve az ütés és színkényszert. Később ez a logika lecserélhető egy fejlettebb, a tökéletlen információs kártyajátékoknál jól bevált Monte Carlo fakeseresésre (MCTS) vagy Minimax algoritmusra, amely valódi kihívást jelenthet a tapasztalt játékosoknak is.
- **Szerveroldali szimuláció:** A Room osztály kiegészülhetne egy dedikált "Bot Room" logikával. Amikor a gép következik, a szerver a játékos lépése után egy emberi gondolkodást szimuláló, rövid késleltetéssel (1-2 másodperc) automatikusan kiszámolná, végrehajtaná és közvetítené a bot lépését a kliens felé, így a hálózati kommunikáció menete szinte változatlan maradhatna.

Irodalomjegyzék

- [1] JGraph Ltd. *draw.io - Diagram Software and Flowchart Maker*. 2026. URL: <https://app.diagrams.net/> (elérés dátuma 2026. 04. 04.).
- [2] Aduasz.com. *Aduász – A kártyajátékok oldala*. URL: <https://aduas.com/> (elérés dátuma 2026. 04. 04.).
- [3] Microsoft Corporation. *Microsoft Word*. Verzió: 2403. 2024. URL: <https://www.microsoft.com/hu-hu/microsoft-365/word>.
- [4] Figma, Inc. *Figma*. URL: <https://www.figma.com/> (elérés dátuma 2026. 04. 05.).
- [5] Sequelize Contributors. *Sequelize - Feature-rich ORM for Node.js*. URL: <https://sequelize.org/> (elérés dátuma 2026. 04. 05.).
- [6] Socket.IO. *Socket.IO - Bidirectional and low-latency communication for every platform*. URL: <https://socket.io/> (elérés dátuma 2026. 04. 05.).
- [7] Tailwind Labs Inc. *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML*. URL: <https://tailwindcss.com/> (elérés dátuma 2026. 04. 05.).
- [8] Cloudflare, Inc. *Cloudflare - The Connectivity Cloud*. <https://www.cloudflare.com/>. Elérve: 2026. 04. 06. 2026.
- [9] Microsoft Corporation. *Microsoft Azure - Cloud Computing Services*. URL: <https://azure.microsoft.com/> (elérés dátuma 2026. 04. 05.).
- [10] PlantUML Team. *PlantUML - Open-source tool for drawing UML diagrams*. URL: <https://plantuml.com/> (elérés dátuma 2026. 04. 05.).
- [11] Wikipedia contributors. *Fisher–Yates shuffle — Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/Fisher-Yates_shuffle (elérés dátuma 2026. 04. 05.).

- [12] DBeaver Corporation. *DBeaver - Universal Database Tool*. URL: <https://dbeaver.io/> (elérés dátuma 2026. 04. 05.).
- [13] OpenJS Foundation. *Express - Node.js web application framework*. URL: <https://expressjs.com/> (elérés dátuma 2026. 04. 05.).
- [14] Auth0. *jsonwebtoken - JSON Web Token implementation (symmetric and asymmetric)*. URL: <https://github.com/auth0/node-jwebtoken> (elérés dátuma 2026. 04. 05.).
- [15] Brian Carlson és a node-postgres közreműködők. *node-postgres - Non-blocking PostgreSQL client for Node.js*. URL: <https://node-postgres.com/> (elérés dátuma 2026. 04. 05.).
- [16] Daniel Wirtz. *bcrypt.js - Optimized bcrypt in plain JavaScript with zero dependencies*. URL: <https://github.com/dcodeIO/bcrypt.js> (elérés dátuma 2026. 04. 05.).
- [17] Troy Goode. *cors - Node.js CORS middleware*. URL: <https://github.com/expressjs/cors> (elérés dátuma 2026. 04. 05.).
- [18] Scott Motte. *dotenv - Dotenv is a zero-dependency module that loads environment variables*. URL: <https://github.com/motdotla/dotenv> (elérés dátuma 2026. 04. 05.).
- [19] Remy Sharp. *nodemon - Monitor for any changes in your node.js application and automatically restart the server*. URL: <https://nodemon.io/> (elérés dátuma 2026. 04. 05.).
- [20] OpenJS Foundation. *Node.js - JavaScript runtime built on Chrome's V8 JavaScript engine*. URL: <https://nodejs.org/> (elérés dátuma 2026. 04. 06.).
- [21] Evan You és a Vite közreműködők. *Vite - Next Generation Frontend Tooling*. URL: <https://vitejs.dev/> (elérés dátuma 2026. 04. 06.).
- [22] Jest Core Team. *Jest - Delightful JavaScript Testing*. URL: <https://jestjs.io/> (elérés dátuma 2026. 04. 06.).

Ábrák jegyzéke

2.1. Oldaltérkép	11
2.2. Főoldal	12
2.3. Szabály oldal	12
2.4. Bejelentkezés oldal	13
2.5. Regisztrációs oldal	13
2.6. Játék oldal	14
2.7. Statisztika oldal	14
2.8. Név megadása	15
2.9. Játék keresés elindítása	16
2.10. Játék keresés	16
2.11. Bejelentkezett keresés	17
2.12. A regisztrációs űrlap	18
2.13. A bejelentkezési felület	19
2.14. Szabályos lépés: a kártya az asztal közepére kerül	20
2.15. Szabálytalan lépés vizuális jelzése és a felugró hibaüzenet	20
2.16. Információs panelek: (balra) az ellenfél következik, (jobbra) a saját állapotunk várakozás közben	21
2.17. Az ütések részletes megjelenítése az egérkurzor rávitelére	22
2.18. A 20/40 bmondást jelző felugró ablak	23
2.19. A Takarás gomb megjelenése a játéktéren	24
2.20. A talon lezárása: az adukártya a pakli tetejére kerül	25
2.21. A takarást megerősítő felugró ablak	25
3.1. Előzetes használatieset-diagram	27
3.2. Bejelentkezés nélküli főoldal drótváz ábrája	29
3.3. Bejelentkezett főoldal drótváz ábrája	29
3.4. Bejelentkezés oldal drótváz ábrája	30
3.5. Regisztráció oldal drótváz ábrája	30

3.6. Játékoldal drótváz ábrája, amikor az ellenfél van soron	31
3.7. Játékoldal drótváz ábrája, amikor mi vagyunk soron	31
3.8. A rendszer három fő komponensének kapcsolata	32
3.9. A Backend belső moduláris felépítésének terve	32
3.10. Tervezett osztálydiagram (UML)	33
3.11. Az adatbázis előzetes logikai terve (ER diagram)	34
3.12. A rendszer fizikai telepítési diagramja (Deployment Diagram)	36
3.13. A rendszer megvalósult osztálydiagramja	38
3.14. A megvalósult adatbázis szerkezete (ER diagram)	40
3.15. Az egységtesztek sikeres lefutása a parancssorban	55

Táblázatok jegyzéke

2.1. A program hardver és szoftver követelményei	9
3.1. A rendszer használati eseteinek részletezése	28
3.2. Az <code>isValidMove</code> metódus működésének részletezése a tényleges algoritmus alapján	41
3.3. Az <code>evaluateStrongerCard</code> metódus működésének részletezése a tényleges algoritmus alapján	42
3.4. A <code>switchTrumpCard</code> metódus működésének részletezése a tényleges algoritmus alapján	42
3.5. A <code>broadcastState</code> metódus működésének részletezése a tényleges algoritmus alapján	43
3.6. A fontosabb REST API végpontok és vezérlők specifikációja	45
3.7. A rendszer használati eseteinek részletezése és teszteredményei	50
3.8. A <code>Card.test.js</code> tesztesetei	55
3.9. A <code>Deck.test.js</code> tesztesetei	55
3.10. A <code>Player.test.js</code> tesztesetei	56
3.11. Az <code>Engine.test.js</code> tesztesetei	57
3.12. A <code>Room.test.js</code> tesztesetei	58

Algoritmusjegyzék

1.	Fisher–Yates (Knuth) keverés algoritmus	39
----	---	----